Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Projekt zespołowy – laboratorium 2

Implementacja bazy danych szeregów czasowych w języku Rust

Architektura systemu i szczegółowy projekt techniczny

Autorzy: Hubert Błonowski
Jan Zaborski

Promotor: mgr inż. Paweł Sobótka

Wersja dokumentu: 1.6
Data: 3 listopada 2026 r.

`https://github.com/janzaborski/rust-tsdb`

Streszczenie

Projekt jednowęzłowej TSDB w Rust obejmuje moduły, model danych, HTTP, podzbiór PromQL i Grafanę. Jeden Index z osobnymi blokadami Series pozwala współbieżnie zapisywać, a rotacja bloków zachowuje dostępność podczas flush. WAL z gałęzi `23-wal-improvements` zachowuje bufor RAM, flusher, okresowy syncer oraz tryby None i Written. Checkpoint wykorzystuje cut() i granicę całych plików. Kontrakty, formaty, indeksy i recovery umożliwiają niezależną implementację magazynu, zapytań i HTTP. To **specyfikacja docelowa**, nie opis prototypu ani wyniki pomiarów. Punktem wyjścia jest lab1; doprecyzowania wymagań wskazano jawnie.

Historia zmian

Wersja	Data	Autorzy	Zakres zmiany
1.0	24.10.2026	H. Błonowski, J. Zaborski	Architektura, API, formaty, diagramy, GUI i plan weryfikacji.
1.1	24.10.2026	H. Błonowski, J. Zaborski	Index/Series, współbieżność bez shardów, rotacja bloków i flush w tle.
1.2	24.10.2026	H. Błonowski, J. Zaborski	Buforowany WAL, bajtowy LSN, poziomy trwałości, checkpoint i recovery.
1.3	02.11.2026	H. Błonowski, J. Zaborski	Krótszy opis; UML przypadków użycia, klas, wdrożenia i stanów.
1.4	02.11.2026	H. Błonowski, J. Zaborski	WAL z brancha 23-wal-improvements; dostosowanie head, checkpointu, recovery, formatów i API.
1.6	03.11.2026	H. Błonowski, J. Zaborski	Ujednolicenie nazw typów, pól, metod i tras HTTP z kodem.

Spis treści

Streszczenie . . . i
Historia zmian . . . i
1 Cel, zakres i decyzje architektoniczne . . . 1
2 Architektura systemu . . . 2
3 Model danych i kontrakty modułów . . . 4
4 Zapis, buforowanie i obsługa opóźnionych danych . . . 8
5 Organizacja danych na dysku . . . 10
6 Checkpoint i odzyskiwanie po awarii . . . 15
7 Język i wykonywanie zapytań . . . 17
8 Komunikacja i publiczne API . . . 20
9 Technologie, konfiguracja i środowisko . . . 23
10 Wizja interfejsu użytkownika i demonstracja . . . 26
11 Weryfikacja architektury i kryteria odbioru . . . 28
A Słownik . . . 31
B Budowanie dokumentu . . . 32
Źródła . . . 32

1 Cel, zakres i decyzje architektoniczne

1.1 Cel, odbiorcy i granice systemu

Baza przyjmuje pomiary aplikacji, przechowuje je we własnym silniku i udostępnia historię oraz agregacje w Grafanie. Projekt dla autorów, promotora i recenzenta określa zachowanie docelowe; przykłady i cele wydajnościowe nie są wynikami prototypu.

Zakres: jednowęzłowy silnik Rust, HTTP, dokładne dopasowanie etykiet, pięć agregacji czasowych i demonstracja w Compose. Grafana i generatory są klientami zewnętrznymi. Poza zakresem pozostają klaster, replikacja, kompresja, retencja, kompaktowanie, alertowanie, pobieranie `/metrics` i własny frontend. Dane rosną do usunięcia środowiska; kopia zapasowa wymaga zatrzymania bazy i skopiowania całego katalogu.

1.2 Doprecyzowania względem laboratorium 1

Wymaganie	Przyjęta interpretacja i konsekwencja
FR-01	Atomowa paczka: błędny punkt odrzuca całość; utrata odpowiedzi po zapisie nie wycofuje danych.
FR-03	Widoczność head poprzedza odpowiedź. None jest domyślne jak w branchu; Written jawnie wymaga pełnego zapisu do OS. Brak gwarancji per żądanie dotyczącej fsync; kryteria trwałości mierzy się w Written.
FR-04	Zakres to obliczenia na siatce czasu. Selektor wybiera ostatnią próbkę; agregacja wymaga funkcji.
FR-08	Czas buforowania biegnie od przyjęcia najstarszej próbki generacji. Flush całej generacji może wcześniej usunąć młodsze dane z RAM; limit pamięci może go przyspieszyć.
FR-09	Out-of-order wyłącznie w otwartym buforze: obowiązują jednocześnie okno opóźnienia i granica zamknięcia.
NFR-03	RSS $\leq$ 1 GiB (nie Gb); 288 MB surowych próbek nie obejmuje zapasowej pojemności, indeksów ani zapytań.
NFR-04	Brak utraty przy awarii procesu dotyczy Written, przy sprawnym OS i nośniku. None dopuszcza utratę i jest mierzone osobno. Błąd I/O WAL kończy proces; awaria hosta/nośnika jest poza gwarancją API.

1.3 Rejestr głównych decyzji

ID	Decyzja, uzasadnienie i koszt
DA-01	Modularny monolit Rust: wywołania wewnątrz procesu upraszczają transakcje i diagnostykę; skalowanie do jednego węzła.
DA-02	Jeden Index, osobna blokada każdego Series: równoległe zapisy rozłącznych szeregów, krótkie blokady katalogu, bez shardów.
DA-03	Posortowane, niekompresowane bloki aktywne i zamrożone: out-of-order i dostępność odczytu podczas flush.
DA-04	Generacje head i jeden flush w tle: bariera zapisu obejmuje drain i sync przez cut(), potem freeze. Zapis segmentu/manifestu odbywa się po jej zwolnieniu.

ID	Decyzja, uzasadnienie i koszt
DA-05	Grafana jako jedyne GUI: standardowe źródło Prometheus, zgodność sprawdzana z przypiętą wersją.
DA-06	Zamknięte zakresy są niezmienne: proste indeksy i rozłączne bloki szeregu; wcześniejszy flush może spowodować odrzucenie punktu.
DA-07	WAL z 23-wal-improvements: bufor RAM, flusher/syncer, watch Written, pliki bez nagłówków i checkpoint po cut()/remove_through().
DA-08	Atomowa paczka i spójny widok wybranych Series: blokady rosnąco po ID; duża operacja może ograniczyć równoległość.
DA-09	None domyślnie, Written per żądanie. Syncer działa okresowo, bez publicznego Synced; LSN jest lokalny dla otwarcia WAL-a.

1.4 Podstawa implementacyjna i zakres dostosowania

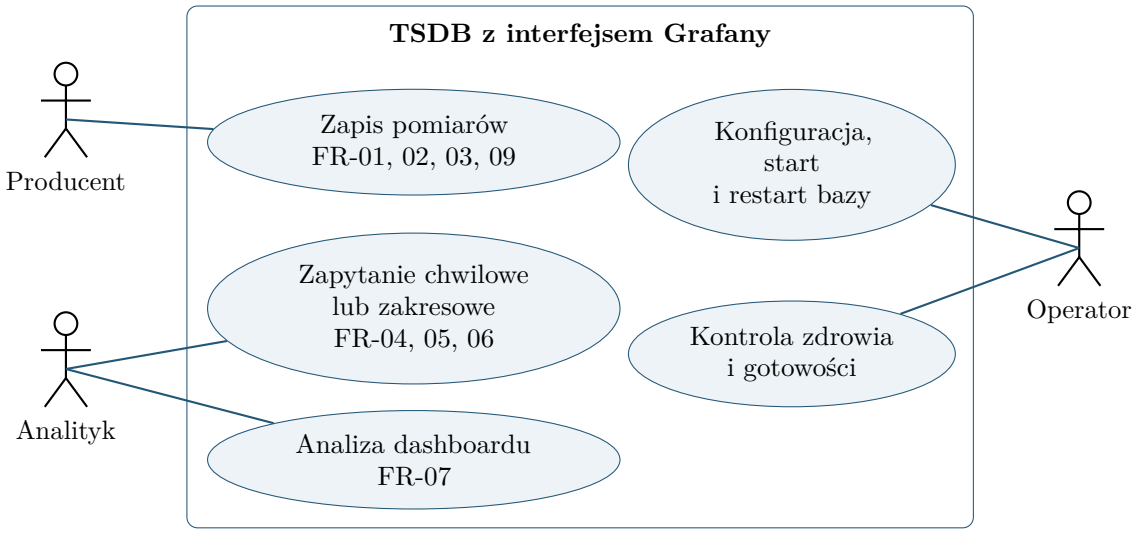
Punktem odniesienia jest lokalny branch `23-wal-improvements`, commit `e1354b9` („docs") [2]. Zachowujemy jego Wal/WalWriter, format kodeka, None/Written, okresowy syncer, cut i ReplaySink. Pozostała część dokumentu opisuje docelowe spięcie tego WAL-a z równoległymi Series i blokami aktywnymi/zamrożonymi; nie przenosi globalnej blokady Db ani obecnego HTTP z brancha.

Wymagane korekty integracyjne zestawia sekcja 5.3. Są projektem do wykonania, a nie informacją, że już istnieją w branchu. W szczególności nie dodaje się trybu Synced, globalnego trwałego LSN ani innego formatu rekordów WAL.

2 Architektura systemu

2.1 Przypadki użycia i aktorzy

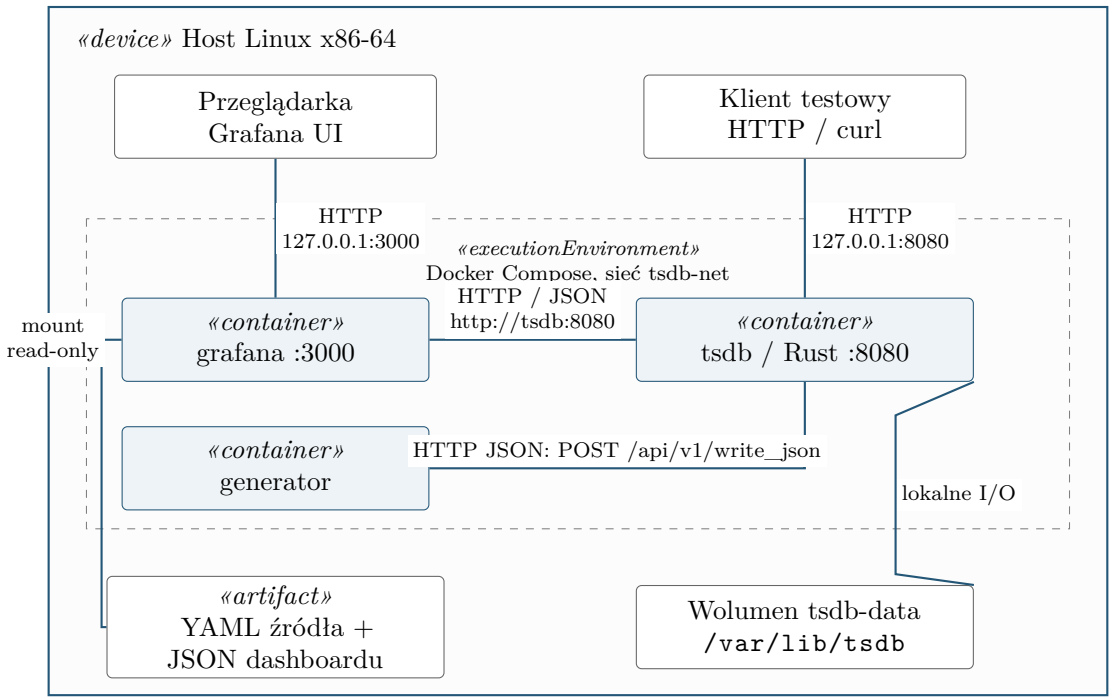
Producent wysyła pojedyncze pomiary lub atomowe paczki. Analityk wybiera czas i serwis w Grafanie albo odpytuje API. Operator uruchamia, konfiguruje i restartuje bazę oraz kontroluje jej stan.



Rysunek 1: Diagram przypadków użycia UML. Flush (FR-08) jest automatycznym mechanizmem silnika, a nie osobnym działaniem użytkownika.

2.2 Wdrożenie i granice procesów

System działa na jednym komputerze. Przeglądarka łączy się z Grafaną; serwer Grafany odpytuje TSDB, a generator wysyła HTTP JSON. W środowisku nie ma serwera Prometheus.



Rysunek 2: Diagram wdrożenia UML z zasobami lokalnymi. Porty hosta są publikowane wyłącznie na loopback; kontenery używają DNS tsdb. Szczegóły dostępu określa sekcja 9.3.

Jeden proces ma wyłączny dostęp do katalogu: systemowa blokada pliku `LOCK` trwa tak długo jak jego otwarty deskryptor; pozostały po awarii plik nie oznacza aktywnej blokady. Sieciowe systemy plików nie są platformą docelową.

2.3 Warstwy, moduły i zależności

Warstwa HTTP odpowiada za protokół, warstwa aplikacyjna za operacje zapisu i zapytań, a silnik za dane i pliki. Parser zależy wyłącznie od typów języka. Silnik nie zna kodów HTTP, Grafany ani składni PromQL.

Moduł	Odpowiedzialność, interfejs i zależności
HTTP	Dekodowanie, limity, błędy i odpowiedzi; Axum/Serde, wywołania zapisu i zapytań.
Model i walidacja	Wspólne typy, kanonizacja klucza, limity nazw/etykiet i wartości; bez HTTP i plików.
Koordynator zapisu	Pula, kolejność blokad, walidacja/rezerwacja; Index, Series, WAL i bariera head.
Index i indeks	Klucz ↔ Series, rejestracja i indeks etykiet; metadane pod krótką blokadą katalogu.
Series i bloki	Posortowane próbki, duplikaty, granice czasu, aktywny/zamrożony blok; blokada per Series, globalny budżet RAM.
WAL	Bufor, lokalny LSN, Written, okresowy sync i rotacja/cut; mutex, flusher/syncer, watch, bez blokad Series w I/O.

Moduł	Odpowiedzialność, interfejs i zależności
Magazyn segmentów	Lokalne pliki, CRC, indeksy, manifest/checkpoint; koordynator flush.
Parser	Tekst → AST/błąd z pozycją; typy języka, bez dostępu do danych.
Wykonawca zapytań	Dobór Series, plan, agregacje i budżety; AST i interfejs odczytu, bez dostępu do WAL.
Uruchomienie i diagnostyka	Konfiguracja, blokada katalogu, recovery, gotowość, logi i zakończenie; zarządza cyklem życia modułów.

2.4 Współbieżność i spójność

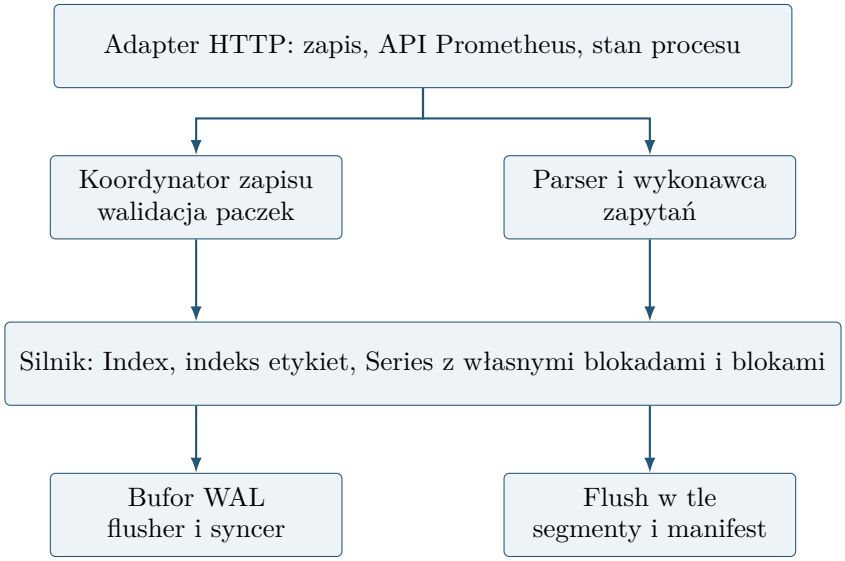
Tokio obsługuje sieć; pula wykonuje pamięciową część zapisu (domyślnie cztery zadania). Każdy Series ma własny RwLock. Katalog zwalnia się przed pobraniem blokad Series.

Operacja	Blokady i punkt publikacji
Zapis	Dla nowych kluczy: mutex rejestracji; potem współdzielona bariera generacji i wyłączne blokady Series rosnąco po ID. Walidacja/rezerwacja → WalWriter.append → publikacja całej paczki przed zwolnieniem którejkolwiek blokady Series.
Append WAL	Thread-local scratch koduje poza mutexem. Mutex bufora chroni kopię ramki i przyrost LSN; brak I/O, head append i wait pod nim. Możliwe oczekiwanie na mutex/budżet, bez lock-free.
Written	Po publikacji zwolnić blokady, barierę i worker; HTTP asynchronicznie wywołuje WalWriter.wait. None odpowiada bez wait. Wątki WAL nie pobierają blokad Series.
ReadView	Pobrać uchwyty i wersję Index, zwolnić katalog, pobrać współdzielone blokady wybranych Series rosnąco po ID (try_read z timeoutem). Zmiana wersji katalogu: zwolnić i ponowić. Skopiować active i uchwyty frozen/dysku, zwolnić Series.
Checkpoint	Wyłączna bariera czeka na enqueue–head append; bez blokad Series wykonuje blokujące Wal.cut, następnie zamraża head i zmienia generację. Sync w cut może wstrzymać zapisy. Odczyty nie pobierają bariery; segment i manifest powstają po jej zwolnieniu (sekcja 6).

Jeden ReadView obejmuje całe query_range, odpowiada chwili stanu wybranych Series i nie zawiera części paczki. Może zobaczyć dane klienta czekającego na Written. Publikacje rozłącznych Series mogą kończyć się poza kolejnością WAL; wspólne Series zachowują tę kolejność. Odczyt po sukcesie widzi zapis w swoim zakresie. Append sam nie wykonuje I/O, ale żądanie może czekać na barierę cut.

3 Model danych i kontrakty modułów

Nazwy odpowiadają kodowi: `Index` (`src/storage/index.rs`), `Series` i `SeriesState` (`src/storage/series.rs`) oraz `LabelSet`, `Sample`, `SeriesId` i `TimeRange` (`src/model.rs`). Bazą zarządza `Db` z `DbState`. Metody to `Db.write(WriteBatch)`, `Db.wait(Lsn)` i `Db.query(&[Matcher],TimeRange)` zwracająca `Vec<SeriesResult>`. Pliki danych obsługuje `SegmentStore` (`src/storage/segment.rs`). Diagram pokazuje istniejące pola i metody oraz planowane bloki.



Rysunek 3: Zależności logiczne. Konfiguracja i diagnostyka są współdzielonymi usługami pomocniczymi.

`MutableBlock`, `FrozenBlock`, `BlockDescriptor`, `ReadView`, granica `sealed_through`, `last_lsn` i wersjonowanie katalogu są elementami projektu do implementacji. Obecny `Series.state` używa `Mutex<SeriesState>`; opisane dalej współdzielone blokady odczytu wymagają docelowo `RwLock`.

3.1 Tożsamość szeregu i reprezentacja punktu

Klucz szeregu składa się z nazwy metryki oraz uporządkowanej listy par etykiet. Klucze etykiet są unikalne; sortowanie jest leksykograficzne według bajtów UTF-8. Nie normalizuje się wielkości liter ani Unicode. Porównanie pełnego klucza rozstrzyga tożsamość również przy kolizji funkcji skrótu.

Typ / pole	Reprezentacja	Reguła
`LabelSet`	Metryka i posortowane pary napisów.	Nazwa metryki: `[a-zA-Z_:][a-zA-Z0-9_:]*`, do 128 bajtów.
`Label`	Nazwa i wartość UTF-8.	Nazwa: `[a-zA-Z_][a-zA-Z0-9_]*`, do 64 bajtów; wartość do 256 bajtów, maksymalnie 16 etykiet.
Nazwy zastrzeżone	Prefiks `__`.	Nie są dopuszczane w etykietach zapisu. `__name__` jest wirtualną etykietą nazwy metryki w odpowiedziach.
`SeriesId`	`u64`, od 0.	Rosnący identyfikator bieżącego head; po restarcie może być inny. Występuje tylko w head i WAL, nie w API, manifeście ani segmentach.
`Sample.timestamp`	`u64`, milisekundy od epoki Unix.	Zakres przyjmowania: 0 do $2^{53} - 1$ ms, aby zachować dokładność całkowitą w klientach JSON.
`Sample`	`timestamp:u64`, `value:f64`.	16 bajtów w bloku; wartość wejściowa musi być skończona. Normalizacja -0 do $+0$.

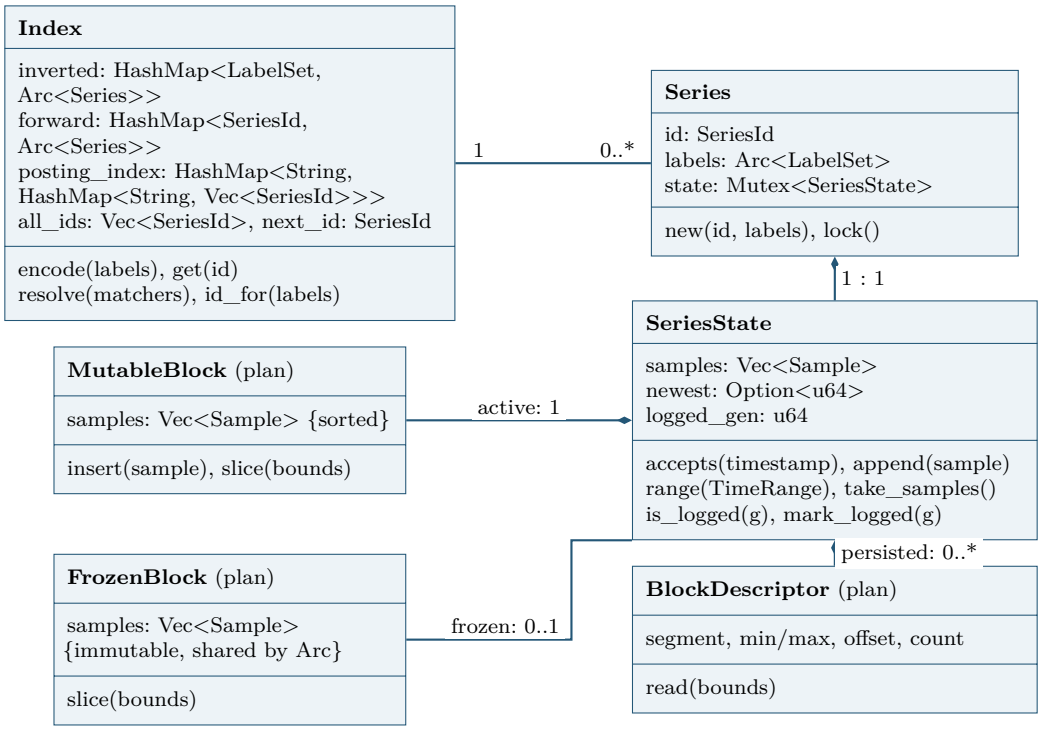
Typ / pole	Reprezentacja	Reguła
`Lsn`	`u64`, lokalny dla Wal::open.	Liczba bajtów ramek przyjętych w tym otwarciu WAL-a; 0 przed pierwszą. Nie przechodzi przez restart i nie jest trwałym kursorem recovery.

Wewnętrzny `LabelSet` zawiera także `__name__` z nazwą metryki. Adapter tworzy go z metryki i etykiet; nazwa zastrzeżona nadal nie jest dozwolona w wejściowym obiekcie labels. WalWriter używa LabelSetEncoding z brancha.

Pusta wartość etykiety jest kanonizowana przez usunięcie tej etykiety. Dzięki temu porównanie z pustym napisem może objąć również szeregi bez danej etykiety. Powtórzona nazwa klucza w obiekcie JSON jest błędem przed kanonizacją, także gdy parser JSON domyślnie potrafiłby ją nadpisać. Nieznane pola wejściowe są odrzucane. Wartości metryk i etykiet w przykładach stosują konwencje Prometheusa; projekt celowo ogranicza dozwolone nazwy do wskazanej składni ASCII.

3.2 Struktury Index, Series i bloki

Inspiracją jest oddzielenie katalogu, synchronizacji szeregu i bloków otwartych/zamkniętych w Gorilla, sekcje 4.2–4.3 [3]. Tutaj jeden Index obsługuje całą bazę, a bloki bez kompresji zapewniają dostępność podczas flush. Buforowany WAL ma jawne poziomy trwałości.



Rysunek 4: Diagram klas modelu head (typy Rust, bez dziedziczenia). Romb oznacza własność; zwykłe linie także współdzielone uchwyty. Klucz: LabelSet, czas: u64; Sample = timestamp: u64 + value: f64 (16 B).

W kodzie `Db.state:RwLock<DbState>` chroni `Index`. `inverted` mapuje LabelSet na szereg, `forward` mapuje SeriesId na szereg, `posting_index` zawiera listy ID według etykiet, a `all_ids` służy do skanowania katalogu. ID nie jest indeksem wektora. Szeregów nie usuwa się, a ID nie wykorzystuje ponownie.

Rejestracja nowych Series wymaga mutexu i ponownego lookup. Przygotowuje je poza publiczną mapą; po udanym append i inicjalizacji head krótka blokada katalogu publikuje Series/indeksy i zwiększa wersję. Katalog nie jest blokowany przez kodowanie WAL ani I/O; nieudana paczka może pozostawić lukę ID, bez pustego publicznego Series.

Każdy Series przechowuje `logged_gen`. Pierwszy zapis w generacji g dołącza do ramki jego definicję, nawet jeśli Series istniał przed cut. Znacznik ustawia się dopiero po append, pod tą samą blokadą Series. Nie tylko create callback decyduje o definicji. Dzięki temu pliki po checkpointcie nie zależą od definicji usuniętych przez remove_through. Zwykła rotacja pliku nie zmienia generacji. Po restarcie Series dostają nowe ID i nieustawiony logged_gen; GC head pozostaje poza MVP.

3.3 Duplikaty i atomowość paczek

W szeregu jest najwyżej jedna wartość dla czasu. Identyczny punkt w active to duplikat bez zmian; inna wartość odrzuca całą paczkę, również dla powtórzeń w niej. Zamknięty zakres odrzuca także identyczny duplikat bez odczytu dysku. Brak bezterminowej idempotencji/rejestru kluczy; po utracie odpowiedzi wynik można sprawdzić zapytaniem.

Walidacja tworzy plan całej paczki; jej definicje i próbki trafiają do jednej ramki. Samych duplikatów nie loguje się. Ich Written czeka na maksimum last_lsn dotkniętych Series, zamiast na globalny koniec bufora zwracany przez pusty append biblioteki. Historyczne dane po otwarciu zaczynają z last_lsn=0, nowe append ustawiają lokalny LSN. Nie jest to obietnica widoczności mniejszych LSN w rozłącznych Series.

3.4 Kontrakty wewnętrzne

Notacja opisuje granice modułów bez narzucania crate'ów. Poza istniejącym API Db i WAL poniższe kontrakty (m.in. submit, snapshot i evaluate) są propozycjami do implementacji. Operacje mogą zawieść; wyjątkiem w branchu jest wait, którego domknięcie błędem wymaga korekty W-04 z sekcji 5.3. Selektory/AST/widoki są pożyczane, QueryBudget mutowalny.

Operacja → wynik	Kontrakt
`canonicalize(Measurement)` → `CanonicalPoint`; `parse(&str)` → `Expr`	LabelSet i walidacja; AST selektora, funkcji albo skalara.
`submit(Vec<CanonicalPoint>,Durability)` → `WriteReceipt`	Widoczna cała paczka i osiągnięty tryb. Receipt: inserted, duplicates, lsn, durability.
`snapshot(Selector,TimeRange,QueryBudget)` → `ReadView`; `read_series(ReadView,SeriesId,TimeRange)` → `SampleCursor`	Wersja katalogu, kopie active, uchwyty frozen/dysku; próbki chronologiczne, bez ujawniania źródła.
`evaluate(Expr,EvalGrid,ReadView,QueryBudget)` → `QueryResult`	Budżet czasu, RAM, próbek i wyniku.
`WalWriter.append(series,samples)` → `Lsn`; `append_samples(samples)` → `Lsn`	Iteratory definicji (ID, LabelSet) i grup (ID, `&[Sample]`). Kodowanie i kopia do RAM bez I/O; błąd WalError.
`WalWriter.wait(Lsn,Durability)`; `written_lsn()`; `backlog_bytes()`	None/Written, watch i pobudzenie flushera; czekanie dopiero po publikacji head.
`Wal.cut()` → `u64`; `remove_through(u64)` → `usize`	Cut drenuje, synchronizuje i rotuje; zwraca ID zamkniętego pliku. Usuwanie tylko po trwałym checkpointcie, nigdy active.

Operacja → wynik	Kontrakt
`freeze_head()` → `FrozenGeneration`; `persist(FrozenGeneration)` → `Checkpoint`	Frozen zawiera g, wal_checkpoint=N, klucze/granice i bloki. Persist poza barierą.
`Wal.open(WalConfig,ReplaySink)`; `Wal.close()`	Odtworzenie przez sink i nowy plik; close drenuje/synchronizuje i dołącza wątki. Filtrowanie po N: korekta W-01.

Błędy modelu/parsera/zapisu/odczytu/zapytań są typowane. WalError zachowuje Io, RecordTooLarge, Corrupt i Closed; przeciążenie jest błędem przyjęcia przed append. Po append publikacja korzysta z rezerwacji, bez zwykłego rollbacku; timeout/anulowanie nie wycofują danych. I/O WAL kończy proces, pozostałe błędy mapuje sekcja 8.5.

3.5 Podział odpowiedzialności zespołu

Osoba	Obszar odpowiedzialności
Jan Zaborski	WAL, segmenty, manifest, indeksy dyskowe, checkpoint i recovery.
Hubert Błonowski	Parser, wykonawca zapytań, adapter HTTP, integracja i dashboard Grafany.
Obaj autorzy	Model danych, bufor i granice jego interfejsu, przegląd kontraktów, testy integracyjne, pomiary i dokumentacja swoich podsystemów.

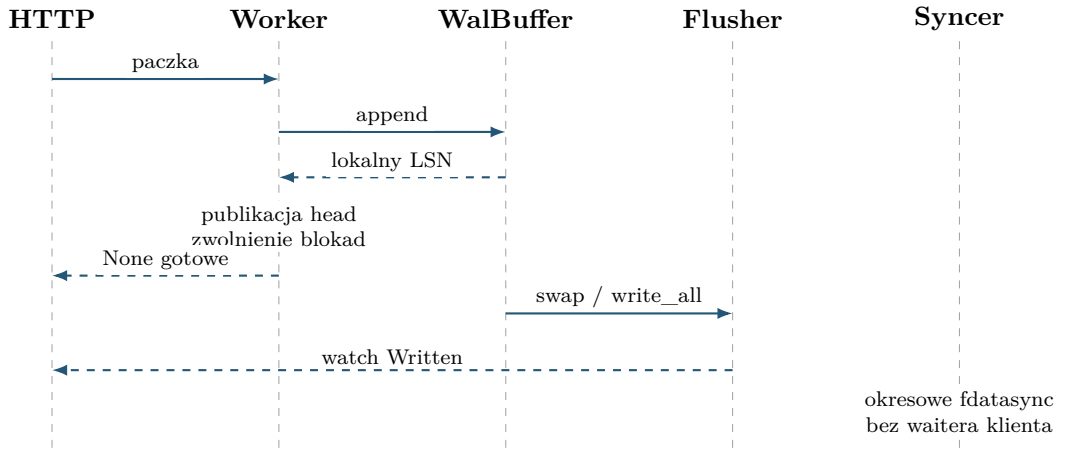
Wykonawcę zapytań można rozwijać z pamięciową implementacją interfejsu odczytu. Magazyn można testować bez HTTP i parsera. Adapter HTTP korzysta z ustalonych typów wyników i błędów. Wspólna akceptacja zmian modelu i kontraktów poprzedza ich wdrażanie po obu stronach.

4 Zapis, buforowanie i obsługa opóźnionych danych

4.1 Algorytm przyjęcia paczki

1. Przed wczytaniem ciała zarezerwować przyjęcie; sprawdzić rozmiar/tryb, zdekodować, kanonizować i pogrupować po Series.
2. Pobrać blokady z sekcji 2.4. Zweryfikować całą paczkę oraz zarezerwować head, indeksy, scratch i miejsce WAL. Brak miejsca: zwolnić blokady/barierę, czekać i walidować ponownie. Paczka niemieszcząca się w pustym budżecie daje 422; przejściowy brak po maks. 1 s daje 503, zawsze przed append.
3. Zebrać definicje Series z logged_gen innym niż g i nowe próbki. WalWriter.append koduje jedną ramkę w scratch i kopiuje ją do WalBuffer.
4. Pod blokadami Series opublikować całość, last_lsn i logged_gen=g. Opublikować nowe Series/indeksy; zwolnić blokady, barierę i worker.
5. None odpowiada; Written czeka asynchronicznie na written_lsn.

Recovery może odtworzyć pełną ramkę bez odpowiedzi lub zakończonego head append. Publikacja i synchronizacja nie tworzą jednej transakcji.



Rysunek 5: Ścieżka zapisu zgodna z WalWriter. None kończy po head append, Written po write_all; okresowy syncer nie potwierdza trybu per żądanie.

4.2 Warunki przyjmowania out-of-order

H_s to największy przyjęty czas szeregu (także na dysku), S_s – największy czas danych zamrożonych/opublikowanych, a W – okno opóźnienia. Dla każdego nowego punktu:

$$t > S_s \quad \wedge \quad t \geq H_s - W.$$

S_s pomija się przed pierwszym zamrożeniem, H_s dla nowego Series. Całą paczkę waliduje się względem H_s sprzed paczki, niezależnie od kolejności JSON; po publikacji H_s jest maksimum starego i nowych czasów. Arytmetyka jest sprawdzana, ujemna dolna granica oznacza zero.

Kolejność decyzji: zamknięty zakres → duplikat/konflikt w otwartym buforze → test W dla nowego punktu. Identyczny otwarty duplikat nie staje się nowym zapisem. Próbka może wyprzedzać zegar serwera najwyżej o 60 s, co chroni H_s przed błędnym przesunięciem. Zegar ścienny służy temu testowi i domyślnemu czasowi zapytania; pobyt w RAM i timeouty używają zegara monotonicznego.

Stan	Czas punktu	Wynik
$H = 12{:}10$, $S = 12{:}00$, $W = 5$ min	12:06 / 12:04	Przyjęcie / przekroczone W.
Po zamrożeniu $S = 12{:}10$	12:09 / 12:11	Zakres zamknięty / nowy aktywny blok (także podczas flush).

Przy $W = 0$ paczka może zawierać kilka czasów, ale żaden nowy nie może poprzedzać starego H_s; punkty paczki nie ustalają kolejności przyjęcia. Profil standardowy: $W = 5$ min.

4.3 Organizacja i limit pamięci

Posortowany wektor dopisuje rosnące czasy na końcu; out-of-order wymaga wyszukiwania binarnego i liniowego przesunięcia. Dwa wyszukiwania wyznaczają wycinek odczytu. Pojemność rośnie przez `try_reserve_exact` w wielokrotnościach 4096 próbek; rozlicza się faktyczną pojemność, także zapas i przejściowe współistnienie starej/nowej alokacji.

Budżet head obejmuje aktywne, zamrożone i podtrzymywane przez ReadView bloki, każdy współdzielony blok raz do ostatniego uchwytu. Katalog, przyjęcia, WAL i zapytania mają osobne budżety. Rezerwacja poprzedza enqueue; błąd nie publikuje próbek. Za mały limit minimalnej alokacji oznacza błąd konfiguracji/paczki, bez nieskończonego flush.

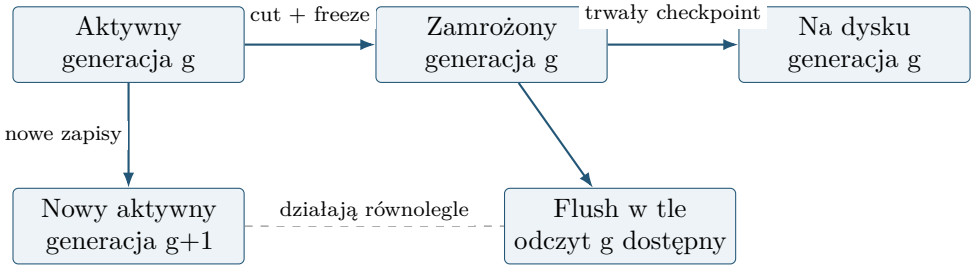
Zamrożenie przenosi wektor do `Arc<FrozenBlock>` bez pełnej kopii. Nowy aktywny blok alokuje w miarę napływu danych. Zapytania kopiują wyłącznie potrzebne aktywne próbki; niezmienne dane

współdzielą. Po publikacji segmentu starszy widok może nadal podtrzymywać RAM, co uwzględnia się w budżecie i pomiarach RSS.

4.4 Wyzwalanie i przebieg zrzutu

Timer co 1 s zleca flush po upływie czasu pobytu najstarszej próbki generacji w RAM; termin dotyczy rozpoczęcia, nie zwolnienia pamięci. Inne wyzwalacze: przekroczenie 80% budżetu lub brak miejsca na paczkę. Pustej generacji nie zrzuca się.

Koordynator wykonuje cut WAL i zamraża razem niepuste bloki. Po zwolnieniu bariery nowe zapisy działają równolegle z jednym flush w tle (sekcja 6). Do jego zakończenia następny flush pozostaje oczekujący, bez kolejki zamrożonych generacji. Przy twardym limicie paczki krótko czekają bez blokad lub dostają 503; dostępność podczas flush podlega ograniczonej pamięci i szybkości dysku.



Rysunek 6: Cut i freeze tworzą nowy active przed zapisem frozen do segmentu. Kolejna rotacja czeka na zakończenie poprzedniego flush.

5 Organizacja danych na dysku

5.1 Katalog i wspólne reguły formatu

```
data/LOCK
data/MANIFEST
data/wal/wal-00000001.log
data/wal/wal-00000002.log
data/segments/segment-00000000000000000001.bin
```

Nazwy WAL mają minimum 8 cyfr jak file_name() w branchu, segmentów danych 20. Ich numeracje są niezależne. Jeden WAL jest aktywny. Pliki WAL nie mają nagłówków ani .tmp; create_new i sync katalogu publikują nowy, początkowo pusty plik. Segmenty/manifest powstają przez .tmp, sync, rename i sync katalogu. Manifest wskazuje segmenty oraz największy zamknięty WAL objęty checkpointem.

Liczby: little-endian, timestamp u64, f64 jako bity IEEE 754, bez paddingu/układu struktur Rust. Napis: u32 długości i UTF-8. Arytmetyka/limity są sprawdzane przed alokacją i odczytem. CRC-32/IEEE [11], zapis u32 LE, wektor `123456789->0xCBF43926`; wykrywanie uszkodzeń, bez uwierzytelnienia.

WAL zachowuje format brancha bez pola wersji. Docelowy indeksowany segment ma wersję 3, manifest 4; ich formaty zastępują wersje z dokumentu 1.3, aby nie utrwalać head SeriesId ani lokalnych LSN. Nie są formatem obecnego SegmentStore JSON z brancha. Nieznana wersja, flagi, UTF-8 i długości zatrzymują start; migracja plików prototypu wymaga osobnego narzędzia.

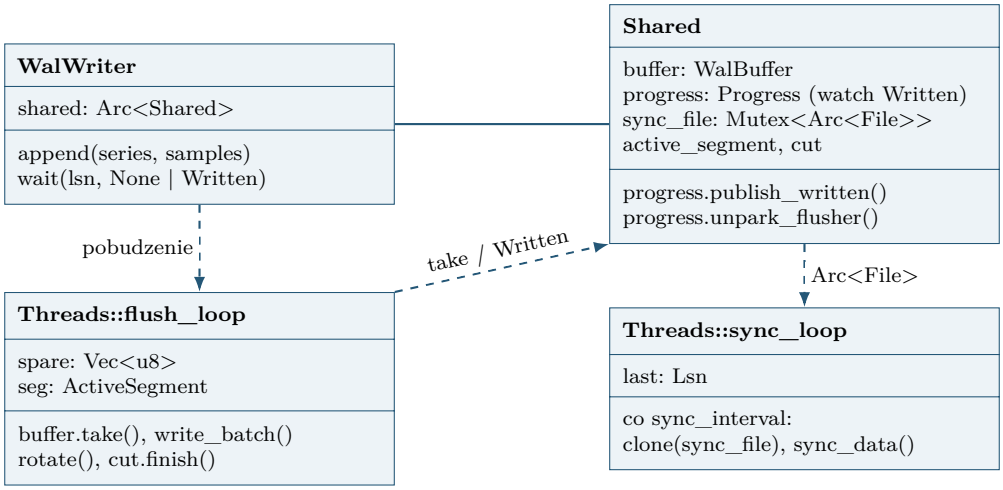
5.2 Bufor WAL, flusher i syncer

Usługa zachowuje `Wal`, klonowalny `WalWriter` i `Shared` z brancha. `WalBuffer` chroni Vec, end i closed jednym mutexem; flusher podmienia Vec z własnym spare. LSN rośnie o pełną długość ramki w jednym otwarciu:

$$0 \le written_lsn \le buffer.end, \qquad backlog_bytes = buffer.end - written_lsn.$$

Po ponownym Wal::open oba liczniki zaczynają od 0. Nie ma synced_lsn ani publicznego oczekiwania na sync.

Tryb	Warunek sukcesu po publikacji head
None (domyślny)	Bez oczekiwania na I/O; awaria procesu może utracić potwierdzoną paczkę. Zachowane pełne ramki mogą wrócić.
Written	`written_lsn()>=lsn`; pełna ramka w OS/page cache. Potwierdzenie przetrwa awarię samego procesu przy sprawnym hoście/nośniku, bez gwarancji dla awarii hosta.



Rysunek 7: Typy i funkcje WAL-a. Flusher i syncer to role wątków Threads. Syncer nie publikuje postępu do klienta; cut obsługuje flusher.

Append. Kodowanie w thread-local scratch, kopia pełnej ramki pod mutexem, zwiększenie end; bez plikowego I/O. Przekroczenie flush_threshold budzi flusher. Pusty append biblioteki zwraca bieżący end; aplikacja obsługuje no-op według sekcji 3. Scratch większy od 1 MiB jest zmniejszany po użyciu.

Flusher. Domyślnie budzony co 100 ms lub po 4 MiB, także przez Written i cut/close. Poza cut/close przy waiterach i buforze mniejszym od commit_bytes=256 KiB zbiera grupę przez 200 μs. Swap pod mutexem, write_all poza nim; Written dopiero po zapisaniu całej przejętej porcji. Krótki write obsługuje write_all [13]. Nowe append używają drugiego Vec.

Syncer. Co 1 s odczytuje Written; przy postępie klonuje Arc aktywnego pliku i wykonuje sync_data. Nie jest pobudzany żądaniem trwałości i nie wysyła potwierdzenia. Rotacja sama synchronizuje stary plik; jego Arc pozostaje ważny podczas równoległego sync.

Rotacja rozmiarowa. Przed zapisem przejętego bufora sprawdza, czy plik już ma co najmniej 128 MiB; jeśli tak, sync starego, create_new następnego i sync wal/. Próg jest miękki: zapisuje cały przejęty bufor, bez dzielenia ramek, i może przekroczyć 128 MiB o rozmiar porcji. Sama rotacja nie zmienia generacji head.

Cut. Serializowany handshake przez mutex/Condvar budzi flusher, który drenuje bufor, synchronizuje/zamyka active i tworzy następny. Zwraca numer zamkniętego pliku. Aplikacja na ten czas

blokuje nowe append barierą generacji, aby cut zakończył drain i dokładnie pokrywał zamrażany head. Wywołanie jest blokujące, poza wątkiem Tokio.

Wait i close. Written rejestruje waiter, subskrybuje ostatni stan watch i budzi flusher; nie wymaga każdego zdarzenia [16]. Obsługa HTTP nakłada timeout 30 s, bez rollbacku. Close odrzuca kolejne append, drenuje, wykonuje końcowy sync i dołącza wątki. Domknięcie kanału nie może udawać sukcesu (W-04).

`writeback=true` uruchamia na Linux `sync_file_range(...,SYNC_FILE_RANGE_WRITE)` po zapisie porcji; na pozostałych platformach to no-op. To optymalizacja writeback, nie fdatasync ani gwarancja trwałości [15]. Błąd write/sync/writeback/tworzenia WAL kończy proces (fail-stop). Sukcesu nie otrzymuje nieosiągnięty cel Written; None mogło odpowiedzieć. Okres 1 s nie ogranicza straty None bez znajomości backlogu i opóźnień I/O.

5.3 Wymagane korekty i kontrakty integracyjne

Poniższe zmiany są uzupełnieniem brancha, nie przebudową jego WAL-a.

ID	Różnica względem e1354b9 i wymagane zachowanie
W-01	Dodać WalConfig.replay_after (u64, domyślnie 0), ustawiane wyłącznie z manifestu na N. Reader/open mają pomijać pliki $\leq N$, także gdy usuwanie przerwała awaria. Nowy plik ma ID większe od N i wszystkich znalezionych plików. Nie dodaje się trwałego bajtowego kursora.
W-02	Zastąpić gałąź readera odcinającą dowolny błąd ostatniego pliku ograniczeniem do niepełnego końca (Truncated). Pełne błędne CRC/kind/payload lub luka plików po N zatrzymują start. Kanoniczność LabelSet sprawdzać przed normalizacją dekodera. Format ramek pozostaje bez zmian.
W-03	Bufor brancha rośnie bez twardego limitu. Dodać rezerwację przed append i limit pojemności active+spare i realokacji (32 MiB); scratch rozliczać w 64 MiB przyjęcia. Początkowe 8 MiB każdego Vec nie jest limitem. Backpressure bez blokad Series.
W-04	Progress.wait nie może ignorować błędu zamknięcia watch. Docelowo `Result<(),WalError>`, z Closed przy zamknięciu kanału; timeout/cancel adaptera nie wycofują head. Zachować jeden watch Written, bez nowej granicy Synced.
W-05	Integracja head stosuje logged_gen i barierę obejmującą cut. Definicja wraca przy pierwszym zapisie Series po cut/restart. Nie wystarcza logowanie tylko nowych szeregów. ReplaySink scala po LabelSet, a nie przez utrwalony head ID.
W-06	Resolver zachowuje buforowanie late series, mapowanie ID i statystyki. Db nie ogłasza gotowości przy dropped_samples > 0, braku budżetu pending lub błędzie semantycznym sinka. Czytnik biblioteczny może raportować osierocone próbki; produkcyjny start nie może ich cicho utracić.

5.4 Format dziennika i lokalny LSN

Plik jest ciągiem ramek od offsetu 0, bez magic, wersji i base_lsn. Ramka dokładnie odpowiada RecordHeader/RecordEncoder brancha:

Offset	Rozmiar	Pole	Znaczenie
0	4 B	len: u32	Długość samego payloadu; maksimum biblioteki 64 MiB.

Offset	Rozmiar	Pole	Znaczenie
4	4 B	crc: u32	CRC(kind || len w LE || payload), w tej kolejności.
8	1 B	kind: u8	Write=0.
9	len B	payload	Dwa liczniki, definicje, grupy próbek.

Cała ramka ma $9 + len$ B. Profil HTTP ogranicza ją dodatkowo do 8 MiB przed append; kodek zachowuje limit payloadu 64 MiB. Pustego rekordu (oba liczniki 0) WalWriter nie dopisuje.

Payload: `series_count:u32`, `samples_count:u32` (liczba grup, nie punktów), następnie:

1. Definicje: `id:u64`, `label_count:u32` i LabelSet: pary napisów (u32 długości + UTF-8), w tym `__name__`.
2. Grupy: `id:u64`, `point_count:u32`, próbki `timestamp:u64,value:f64` po 16 B. Pusta grupa jest pomijana przez encoder.

Aplikacja porządkuje definicje/grupy po ID i punkty po czasie, choć WAL nie wymaga sortowania grup. Dekoder wykrywa nadmierne liczniki, UTF-8, brakujące i nadmiarowe bajty; adapter dodatkowo waliduje etykiety, czasy, skończoność i konflikty. Jednostka replay to pełna ramka, bez osobnego rekordu commit.

Próbka istniejącego Series: payload $8 + 12 + 16 = 36$ B, ramka 45 B. Pierwsza paczka API dwóch punktów nowego Series ma LabelSet 74 B, definicję $8 + 74 = 82$ B, payload $8 + 82 + 12 + 32 = 134$ B i ramkę 143 B. Jeśli jest pierwszą po otwarciu, LSN=143 i EOF=143. Po rotacji LSN nadal rośnie; po restarcie zaczyna od 0 niezależnie od EOF starszych plików. Dlatego LSN z odpowiedzi nie porównuje się między uruchomieniami ani nie zapisuje w manifeście.

5.5 Segment danych

Docelowy segment przechowuje klucze, nie head SeriesId, tak jak DiskSeries w branchu. Własny indeksowany format ułatwia odczyt wycinków: nagłówek 48 B, słownik LabelSet, bloki 16-bajtowych próbek, indeks i CRC całego pliku (bez końcowego pola CRC). Jeden flush obejmuje niepuste frozen generacji.

Offset	Rozmiar	Pole	Znaczenie
0	8 B	magic	ASCII `RSTSEG03`.
8 / 10 / 12	2 / 2 / 4 B	version / flags / header_len	u16=3 / u16=0 / u32=48.
16	8 B	generation	Numer generacji head, u64.
24	8 B	wal_checkpoint	ID zamkniętego WAL N z cut, nie LSN.
32	8 B	series_count	Liczba kluczy/bloków/wpisów indeksu.
40	8 B	index_offset	Początek indeksu od początku pliku.

Słownik od offsetu 48 zawiera LabelSetEncoding dla każdego bloku, w kolejności kanonicznych kluczy; indeks wskazuje początek danego klucza. Po słowniku następują w tej samej kolejności niepuste bloki próbek `timestamp:u64,value_bits:u64`, czasy ściśle rosnące. Indeks ma po 48 B na blok:

Offset wpisu	Rozmiar	Pole	Znaczenie
0	8 B	key_offset	Offset LabelSet w słowniku; nie identyfikator Series.
8 / 16	8 / 8 B	min_time / max_time	Pierwszy/ostatni czas, u64.
24 / 32	8 / 8 B	data_offset / point_count	Offset i liczba próbek, u64.
40 / 44	4 / 4 B	block_crc32 / reserved	CRC bloku / zero, u32.

Dla N_p punktów, K bloków i D bajtów słownika:

$$index_offset = 48 + D + 16N_p, \qquad file_size = 48 + D + 16N_p + 48K + 4.$$

Dwa punkty przykładowej metryki: $D = 74$, plik 206 B, dane od 122, indeks od 154, końcowe CRC od 202.

Start kontroluje długości, sumę pliku, klucze, offsety, kolejność czasów i zgodność min/max; opcjonalnie CRC całego czytanego bloku. Słownik/indeks pozostają w RAM. Deskryptory są wiązane z bieżącym head przez LabelSet; próbki czyta się pozycyjnie/binarne wyszukiwanie, bez wczytania całego Series. Pliki pozostają niezmienne.

5.6 Manifest i katalog szeregów

MANIFEST ma kopertę: magic `RSTMAN04` (8 B), version u16=4, flags u16=0, json_len u32; potem JSON UTF-8 (do 64 MiB) i CRC nagłówka z payloadem. Bez paddingu i dodatkowych bajtów po CRC. Przykład po pierwszym cut i flush paczki API; crc32=0 jest miejscem na rzeczywistą sumę segmentu:

```
{
  "format_version": 4, "generation": 1, "wal_checkpoint": 1,
  "segments": [{
    "generation": 1, "file": "segment-00000000000000000001.bin",
    "wal_checkpoint": 1, "size_bytes": 206, "crc32": 0
  }],
  "series": [{
    "metric": "temperature_celsius",
    "labels": [["instance", "1"], ["service", "api"]],
    "sealed_through": 1792843201000
  }]
}
```

Checkpoint N oznacza: wszystkie próbki z plików WAL $\leq N$ odwzorowano w segmentach. Recovery pomija te pliki nawet przed ich usunięciem i czyta pliki $> N$ od offsetu 0. Nie ma kotwicy, offsetu replay, checkpoint_lsn ani next_series_id. Generacja head i ID pliku są niezależne.

Katalog/granice pochodzą z freeze, bez późniejszych Series/active. Szeregi są uporządkowane po kanonicznym kluczu, segmenty po generacji; nazwy plików są pojedynczymi komponentami ścieżki. CRC segmentu powtarza jego końcową sumę; JSON nie narzuca kolejności pól, liczniki dekoduje się bez f64. Początek: generation=0, wal_checkpoint=0 i puste tablice, WAL 1.

Walidacja sprawdza wersję, CRC, rozmiary, rosnące generacje/checkpointy, unikalne klucze i rozłączne zakresy bloków danego klucza. Każdy Series katalogu ma blok; sealed_through to maksimum na dysku, początkowo $H_s = S_s$. Po odbudowie Index przydziela świeże SeriesId.

5.7 Indeksy w pamięci i plan odczytu

Index (rys. 4) używa odwróconych, uporządkowanych list ID dla metryki i par etykiet. Wybór zaczyna od metryki, przecina listy matcherów i sprawdza pełny klucz. Pusty matcher wybiera brak etykiety; pustych wartości nie trzeba indeksować.

Deskryptory Series zawierają segment, min/max, offset i liczbę punktów. Reguła $t > S_s$ zapewnia rozłączne zakresy: deskryptory eliminują zbędne segmenty, binarne wyszukiwanie wybiera wycinki próbek. Indeksy odbudowuje silnik, bez zewnętrznej bazy. Ich RAM zależy od liczby Series, długości etykiet i segmentów; bez kompaktowania liczba deskryptorów rośnie.

6 Checkpoint i odzyskiwanie po awarii

6.1 Trwałe opublikowanie generacji

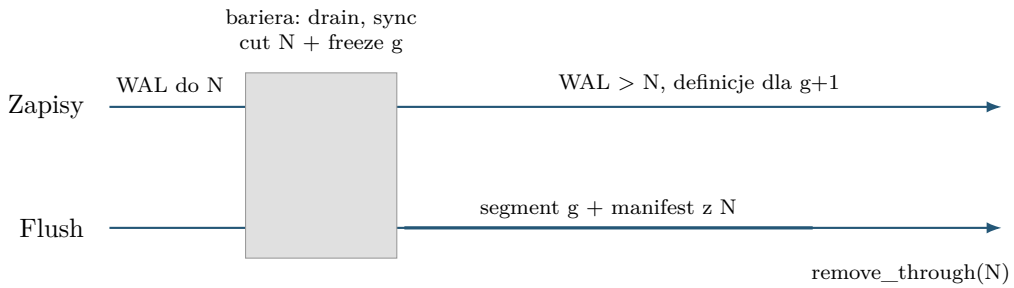
Jeden koordynator wykonuje poniższy protokół. g numeruje head, N jest ID zamkniętego pliku WAL; lokalny LSN nie wyznacza checkpointu.

Faza	Operacje w kolejności
Bariera zapisu	Uzyskać wyłączną barierę; dokończyć rozpoczęte append–head append, bez czekania na klientów Written. Bez blokad Series wywołać Wal.cut(). Brak nowych append pozwala flusherowi opróżnić bufor, sync starego, utworzyć nowy i zwrócić N. Na tym etapie zapisy czekają na I/O; odczyty nie pobierają tej bariery.
Freeze	Nadal pod barierą przenieść active do frozen g pod krótkimi blokadami Series, podnieść S_s, zapamiętać katalog po kluczach. Uruchomić $g + 1$; logged_gen starszej generacji wymusi nowe definicje. Zwolnić barierę. Następne append trafiają do WAL o numerze $> N$.
Segment w tle	Zapisać segment g z wal_checkpoint=N; sync pliku → rename → sync segments/. Nowe active przyjmują dane, frozen pozostają czytelne.
Manifest	Zapisać/sync MANIFEST.tmp z katalogiem freeze, g, N i segmentami; atomowy rename i sync katalogu danych. To punkt publikacji checkpointu. Nie dołączać szeregów utworzonych podczas flush.
Podmiana head	Pod krótkimi blokadami Series podmienić frozen na deskryptory. Nie nadpisywać nowszych active, H_s, last_lsn ani logged_gen. ReadView zachowują uchwyty.
Usunięcie WAL	Dopiero teraz Wal.remove_through(N); metoda dodatkowo chroni active i synchronizuje katalog po usunięciu. Zwolnić własne uchwyty RAM. Żaden plik $> N$ nie podlega temu usunięciu.

Platforma docelowa: Linux i lokalny system plików z atomowym rename oraz sync plików/katalogów [14]. Written nie chroni przed awarią hosta. Zakończony checkpoint i close opierają się na gwarancjach fsync/fdatasync, bez ochrony przed fizycznym uszkodzeniem nośnika.

6.2 Algorytm uruchomienia

1. Sprawdzić konfigurację i uzyskać LOCK; ready=false. W pustym katalogu trwale utworzyć WAL 1 i początkowy manifest z $N = 0$. Przerwaną inicjalizację wolno dokończyć tylko bez danych/segmentów, z poprawnym pustym WAL. Inny brak manifestu jest błędem, bez resetu.



Rysunek 8: Granica checkpointu pokrywa całe pliki WAL. Pauza obejmuje I/O cut, ale zapis segmentu i manifestu jest poza barierą.

2. Sprawdzić manifest i wskazane segmenty, odtworzyć Index po LabelSet, $H_s = S_s$, last_lsn=0 i nieustawione logged_gen. Nie dołączać segmentów spoza manifestu.
3. Użyć poprawki W-01: pomijać WAL $\leq N$ niezależnie od ich obecności. WAL $N + 1$ musi istnieć, choćby pusty; do najwyższego h wymagać ciągłej numeracji. Czytać od offsetu 0 w kolejności plików/ramek; błędy długości, CRC, kind i dekodowania obsługuje W-02.
4. Resolver mapuje ID z logu na bieżące ID zwracane przez ReplaySink.series(LabelSet). Różne ID z tym samym kluczem trafiają do jednego Series. Próbki wcześniejsze od definicji są przechowywane w pending i dostarczane po niej.
5. Sink odtwarza próbki chronologicznie w active generacji $g + 1$, sprawdza semantykę i konflikty. Identyczne powtórzenia pomija. Punkt w zakresie już pokrytym segmentem jest błędem integracji; nie jest ponownie zapisywany ani cicho gubiony. Nie ponawia się testów bieżącego zegara ani zmienionego okna W.
6. Niepełny koniec tylko ostatniego WAL: truncate do ostatniej pełnej ramki i sync. Pełna ramka z błędnym CRC/kind/payload zatrzymuje start także na końcu. Resolver.finish raportuje late/unknown/dropped; niezerowe dropped lub błąd sinka blokują gotowość (W-06).
7. Jak w Wal::open, utworzyć świeży WAL o ID $\max(h, N) + 1$, z create_new i sync katalogu; nie dopisywać do starego. Nowy WalBuffer i watch zaczynają od LSN=0. Index przydziela kolejne ID ponad największe *bieżące* ID; nie zachowuje ID z logu.
8. Po walidacji usunąć .tmp, sieroty segmentów i WAL $\leq N$. Uruchomić wątki WAL i gotowość. Bufor replay i pending podlegają budżetowi pamięci; za mały limit zatrzymuje start. Odzyskane próbki od razu kwalifikują się do czasowego flush.

6.3 Scenariusze awarii i granice gwarancji

Moment awarii	Recovery i znaczenie dla klienta
Przed append	Brak ramki i sukcesu.
Po append, przed head	Pełna zapisana ramka może wrócić, także bez odpowiedzi.
None przed write	Paczka może zniknąć; dopuszczona słabsza trwałość.
Written przed okresowym sync	Awaria samego procesu zachowuje ramkę; awaria hosta może ją utracić.
Niepełna ostatnia ramka	Tylko niepełny koniec ostatniego pliku jest odcinany; bez Written dla tej ramki.
Podczas cut / przed segmentem	Stary manifest obowiązuje. Replay obejmuje też rotacje wykonane przez niedokończony cut.

Moment awarii	Recovery i znaczenie dla klienta
Po rename segmentu, przed manifestem	Segment jest sierotą; replay według starego N, bez podwójnego odtwarzania.
Po manifeście, przed/podczas usuwania	WAL $\leq N$ jest pomijany, nawet jeśli jeszcze istnieje; wszystkie $> N$ pozostają do replay.
Definicja nigdy nie nadeszła	Resolver raportuje dropped, a start produkcyjny kończy się błędem, bez cichego sukcesu.

Błąd segmentu zatrzymuje przyjęcia, zachowując head/WAL i odczyt do diagnostyki/restartu. I/O WAL: fail-stop. Po nieopublikowanym freeze S_s może wrócić do poprzedniego checkpointu; nie narusza to gwarancji Written.

6.4 Cykl życia procesu

SIGTERM zamyka przyjęcia i kończy części pamięciowe zadań. Koordynator kończy rozpoczęty cut przed close i nie rozpoczyna nowego; cut i close nie mogą ścigać się o zamknięcie flushera. Po osiągnięciu Written dla oczekujących klientów wywołuje Wal.close() w zadaniu blokującym: zapisuje także None, wykonuje końcowy sync i dołącza wątki. Flush danych kończy lub bezpiecznie przerywa, zachowując pliki replay; pełny flush head nie jest wymagany. Limit 30 s może skończyć się wymuszonym zatrzymaniem i zwykłymi gwarancjami None/Written. Drop biblioteki próbuje zamknąć WAL, ale poprawny shutdown jawnie sprawdza wynik close.

7 Język i wykonywanie zapytań

7.1 Obsługiwana składnia

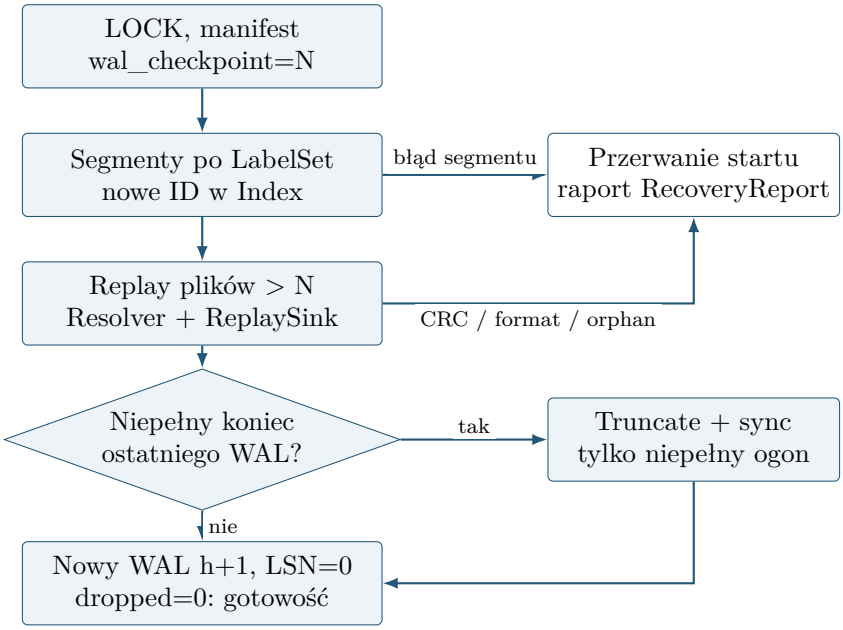
Obsługiwany jest nazwany selektor z dokładnymi matcherami, jedna funkcja czasowa na szereg oraz skalar (np. 1+1 do próby połączenia). Gramatyka:

```
expr       = selector | function "(" selector "[" duration "]" ")"
           | scalar ;
selector   = metric [ "{" [ matcher { "," matcher } ] "}" ] ;
matcher    = label "=" string ;
function   = "avg_over_time" | "min_over_time" | "max_over_time"
           | "sum_over_time" | "count_over_time" ;
duration   = positive_integer unit ;
unit       = "ms" | "s" | "m" | "h" | "d" | "w" ;
scalar     = number [ "+" number ] ;
```

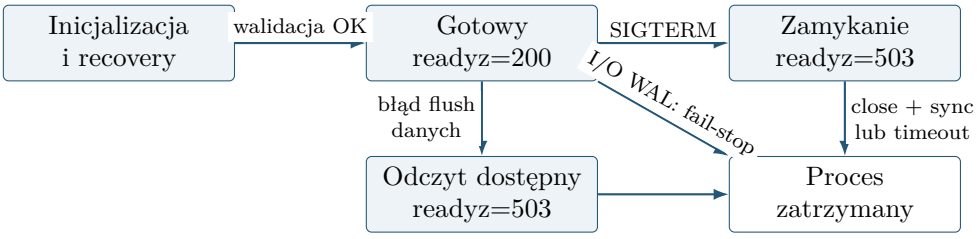
Nazwy: sekcja 3; napisy w podwójnych cudzysłowach z escape JSON; number to skończone f64 o składni JSON. Whitespace między tokenami jest ignorowany, parser zużywa całe wejście. Powtórzony matcher i etykieta z prefiksem `__` są błędem; nazwę metryki podaje się wprost. Maksimum tekstu: 16 KiB.

```
temperature_celsius
temperature_celsius{service="api",instance="1"}
avg_over_time(temperature_celsius{service="api"}[5m])
count_over_time(temperature_celsius[30s])
1+1
```

Odrzucane: regex, matchery inne niż =, selektor bez nazwy, samodzielny range vector, podzapytania, offset, @, zagnieżdżenia i agregacje między Series (np. `sum(temperature_celsius)`). Tokenizator



Rysunek 9: Recovery stosuje granicę plików i Resolver z brancha. Nie wymaga trwałego LSN ani utrwalonych head SeriesId.



Rysunek 10: Diagram stanów procesu. Błąd startu kończy proces bez gotowości; ze stanu ograniczonego odczytu operator może wykonać restart/zamknięcie.

i parser rekurencyjny tworzą AST, bez generatora pełnego PromQL. Walidacja sprawdza nazwy, dodatnie/mieszczące się okno i typ wyniku; błąd zawiera offset bajtowy i opis.

7.2 Czas obliczenia i wybór próbek

Instant oblicza wyrażenie w T, range na siatce

$$T_k = start + k \cdot step, \qquad k = 0, \ldots, \lfloor (end - start)/step \rfloor .$$

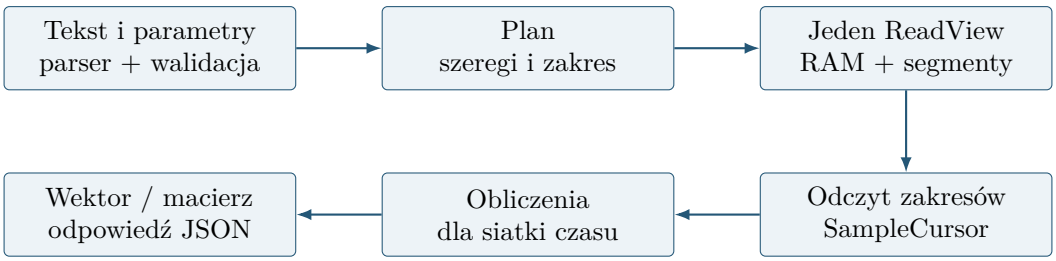
Wymagania: $step > 0$, $start \leq end$, sprawdzana arytmetyka całkowitych milisekund. Koniec wchodzi do wyniku tylko na siatce.

Wyrażenie	Semantyka dla każdego pasującego szeregu
Selektor	Najnowsza próbka w $[T - D, T]$, domyślnie $D = 5$ min (lookback). Bez interpolacji, agregacji i markerów staleness; brak próbki pomija Series. Timestamp wyniku to T, nie czas pomiaru; zachowuje `__name__`.
Funkcja z R	Okno $(T - R, T]$: lewa granica wykluczona, prawa włączona [5]; lookback nie ogranicza okna. Zachowuje etykiety, usuwa nazwę metryki. Puste okno pomija Series dla wszystkich funkcji, także count (brak danych nie oznacza zera).
`avg_over_time`	Średnia arytmetyczna, równa waga próbek.
`min_over_time`, `max_over_time`	Minimum / maksimum.
`sum_over_time`	Suma w chronologicznym porządku.
`count_over_tim e`	Liczba próbek jako wartość liczbowa.

Funkcje dotyczą próbek liczbowych [6]. Obliczenia f64 używają sumowania kompensowanego; średnia = suma/liczność. Przy przepełnieniu sumy średnią przelicza się ze składników x_i/n. NaN i nieskończoności wyników koduje się jako napisy JSON.

Przykład granic: punkty (czas w s, wartość): $(0, 10), (60, 20), (120, 30), (180, 40)$. Dla `avg_over_time(metric[2m])`: $T = 120 \Rightarrow (0, 120] \Rightarrow 25$; $T = 180 \Rightarrow (60, 180] \Rightarrow 35$; $T = 300 \Rightarrow (180, 300] \Rightarrow$ brak elementu. Zakres $[120, 190]$, krok 60 s oblicza tylko 120 i 180. Przeniesienie próbek do segmentu nie zmienia wyniku.

7.3 Plan wykonania i limity pracy



Rysunek 11: Przepływ zapytania. Każdy etap może zakończyć się błędem lub przekroczeniem budżetu.

Plan pobiera $[start - D, end]$ dla selektora lub $(start - R, end]$ dla funkcji, z bezpiecznym przycięciem do dziedziny czasu. ReadView raz kopiuje potrzebne active, współdzieli frozen; dysk czyta blokowo

(I/O 64 KiB). Przesuwane okno utrzymuje sumę/liczność, a min/max kolejki monotoniczne, bez ponownego odczytu całego zakresu w każdym kroku. Nieskończony stan sumy odbudowuje się z bieżącego okna, aby skrajna wartość po jego opuszczeniu nie zatruwała następnych wyników.

Budżet zapytania: 1000 Series, 2 mln różnych wejściowych próbek, 1 mln punktów wyniku łącznie, 64 MiB i 30 s. RAM obejmuje metadane widoku, kopie active, okna i wynik przed serializacją; wspólne frozen rozlicza globalny budżet head, także podtrzymywane tylko przez ReadView. Odczyt strumieniowy nie znosi limitu próbek. Przekroczenie oznacza błąd bez częściowego wyniku; timeout sprawdza się też w pętlach obliczeń/I/O, rozłączenie sygnalizuje anulowanie.

Wyniki są leksykograficzne po kanonicznym kluczu, punkty chronologiczne. Wartości f64 to napisy, timestampy liczby sekund Unix. Instant scalar to para czasu/wartości; range scalar to macierz jednego szeregu o pustych etykietach.

8 Komunikacja i publiczne API

Nazwy istniejących tras w `src/api.rs` to `POST/api/v1/write_json` (handler `write_json`) i `GET/api/v1/read` (handler `read`). Obecny zapis przyjmuje tablicę obiektów `labels`/`samples`, gdzie próbka ma pola `t`/`v`; odczyt używa `start`, `end`, `name` i filtrów etykiet. Poniżej opisano docelowe rozszerzenie kontraktu zapisu oraz planowane endpointy Prometheusa i diagnostyki.

8.1 Transport i konwencje

HTTP/1.1 z keep-alive, UTF-8, JSON `application/json`; bez WebSocketów, brokera i skompresowanych żądań. Profil zapytań przejmuje endpointy, kopertę, wektory i macierze Prometheusa [4]; nie zapewnia pełnego PromQL/Prometheusa. Własny zapis nie jest Remote Write ani OpenMetrics.

8.2 Zapis pojedynczy i paczkowy

`POST/api/v1/write_json` przyjmuje dokładnie jeden z dwóch wariantów: obiekt pojedynczego pomiaru albo obiekt zawierający wyłącznie niepustą tablicę `points`. Każdy pomiar wymaga pól `metric`, `timestamp` i `value`; brak `labels` oznacza pusty zbiór. `timestamp` jest całkowitą liczbą **milisekund**, a `value` liczbą JSON. W jednej paczce wolno przesłać do 10 000 punktów. Maksymalny rozmiar surowego żądania wynosi 4 MiB. Opcjonalny parametr query `durability=none|written` nadpisuje ustawienie procesu; domyślnie jest to none. Wartość synced jest nieobsługiwana (400). Powtórzony parametr, nieznana nazwa albo wartość jest błędem 400. Tryb dotyczy całej paczki i nie zmienia jej atomowości.

```
POST /api/v1/write_json?durability=written
Content-Type: application/json

{
  "metric": "temperature_celsius",
  "labels": {"service": "api", "instance": "1"},
  "timestamp": 1792843200000,
  "value": 24.5
}
```

Wariant paczkowy:

```
{
  "points": [
    {
      "metric": "temperature_celsius",
      "labels": {"service": "api", "instance": "1"},
      "timestamp": 1792843200000,
```

```
      "value": 24.5
    },
    {
      "metric": "temperature_celsius",
      "labels": {"service": "api", "instance": "1"},
      "timestamp": 1792843201000,
      "value": 25.5
    }
  ]
}
```

Odpowiedź dla pierwszej paczki dwóch nowych punktów po otwarciu, przy `durability=written`:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "status": "success",
  "data": {"inserted": 2, "duplicates": 0, "lsn": 143, "durability": "written"}
}
```

`inserted+duplicates` = liczba punktów wejścia. LSN to koniec ramki w bieżącym Wal::open, nie trwały identyfikator żądania; po restarcie może się powtórzyć. Durability potwierdza osiągnięty tryb. Gwarancje określa sekcja 5.2, a reguły duplikatów i atomowości sekcja 3. Przed enqueue błąd odrzuca całą paczkę; po enqueue timeout/anulowanie/ utrata połączenia nie wycofują zmian. Timeout daje 503/timeout z niepewnym wynikiem: WAL może osiągnąć wymagany poziom później.

8.3 Zapytania chwilowe i zakresowe

Obsługiwane są metody GET oraz POST z `application/x-www-form-urlencoded`. Nie wysyła się JSON jako ciała zapytania Prometheus. Parametry w danym żądaniu pochodzą albo z query string, albo z formularza; duplikaty parametrów i mieszanie obu źródeł są błędem. Całość parametrów nie może przekraczać 32 KiB.

Endpoint	Parametry	Znaczenie
`/api/v1/query`	`query`; opcjonalnie `time`, `timeout`	`time` domyślnie równe chwili odebrania żądania.
`/api/v1/query_range`	`query`, `start`, `end`, `step`; opcjonalnie `timeout`	Obliczenie na siatce czasu; wszystkie cztery główne parametry są wymagane.

Czasy przyjmują sekundy Unix z częścią ułamkową lub RFC 3339 ze strefą. Po konwersji muszą mieścić się w zakresie czasu z sekcji 3. Wymagana jest dokładność co najwyżej milisekundowa; dodatkowe niezerowe cyfry są odrzucane zamiast cichego zaokrąglania. Krok przyjmuje dodatnią liczbę sekund, np. `0.5`, albo pojedynczą jednostkę jak `15s`. Okno i timeout przyjmują dodatnią liczbę całkowitą z jedną jednostką z gramatyki `duration`. Timeout efektywny jest mniejszą wartością limitu serwera i parametru klienta.

Nieznane parametry zwracają 400; nie są ignorowane. Parametry opcjonalne pełnego API Prometheusa, np. `lookback_delta` i `limit`, pozostają nieobsługiwane w tym profilu. Grafana korzysta z konfiguracji nieprzesyłającej tych rozszerzeń.

```
GET /api/v1/query?query=temperature_celsius&time=1792843201

{
```

```
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {"__name__": "temperature_celsius", "service": "api", "instance": "1"},
        "value": [1792843201, "25.5"]
      }
    ]
  }
}
```

Przykład odpowiedzi zakresowej dla tej samej metryki od 1792843200 do 1792843201 z krokiem 1 s:

```
{
  "status": "success",
  "data": {
    "resultType": "matrix",
    "result": [
      {
        "metric": {"__name__": "temperature_celsius", "service": "api", "instance": "1"},
        "values": [[1792843200, "24.5"], [1792843201, "25.5"]]
      }
    ]
  }
}
```

Brak danych oznacza `result:[]` z HTTP 200. Wynik `1+1` w zapytaniu chwilowym ma `resultType:"scalar"` oraz `result:[1792843201,"2"]` dla powyższego czasu. Wartości specjalne wyników obliczeń koduje się jako `"NaN"`, `"+Inf"` lub `"-Inf"`. Wejście zapisu nadal ich nie dopuszcza.

8.4 Metadane i stan serwera

Metoda i ścieżka	Kontrakt profilu
GET `/api/v1/labels`	Posortowana lista nazw etykiet oraz `__name__` z katalogu.
GET `/api/v1/label/<name>/values`	Posortowane unikalne wartości etykiety; dla `__name__` nazwy metryk. Nieistniejąca poprawna nazwa daje pustą listę.
GET `/api/v1/status/buildinfo`	Koperta sukcesu z `data.version`, `data.revision`, `data.branch`, `data.buildDate` i `data.implementation`. To dane kompilacji TSDB; nie podszywają się pod wersję Prometheusa.
GET `/healthz`	200 i `ok` jako tekst, jeśli działa pętla obsługi procesu.
GET `/readyz`	200 i `ready` po recovery i przy sprawnym zapisie; 503 i tekst przy braku gotowości.

Odpowiedzi metadanych stosują kopertę `status:"success",data:[...]`. Endpointy etykiet nie przyjmują w tym profilu filtrów czasu ani `match[]`; operują na całym katalogu. Nie obsługuje się `/series`, `/metadata`, reguł, exemplarów ani administracyjnego usuwania danych. Dashboard nie wymaga tych operacji.

8.5 Błędy i przeciążenie

HTTP	errorType	Zastosowanie
400	`bad_data`	Błędny JSON, parametry, składnia, nazwy, typy albo nieobsługiwana konstrukcja języka.
404	`not_found`	Nieznany endpoint.
405	`bad_data`	Niewłaściwa metoda; nagłówek `Allow`.
409	`conflict`	Różne wartości dla tego samego czasu i szeregu.
413	`too_large`	Za duże ciało, tekst zapytania lub liczba punktów paczki.
415	`bad_data`	Nieobsługiwany typ treści lub kodowanie wejścia.
422	`execution`	Zamknięty zakres, przekroczone okno opóźnienia, limit szeregów, pamięci lub rozmiaru wykonywanego zapytania.
503	`unavailable` / `timeout`	Pełna kolejka, zajęte miejsca zapytań, brak gotowości, timeout albo zatrzymany zapis po błędzie I/O.
500	`internal`	Nieoczekiwany błąd lub uszkodzenie podczas odczytu; szczegóły ścieżek i systemu trafiają do logów.

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{
  "status": "error",
  "errorType": "execution",
  "error": "point belongs to a sealed time range",
  "details": {"reason": "sealed_range", "point_index": 1}
}
```

`details` jest rozszerzeniem własnego endpointu zapisu; indeks jest liczony od zera w oryginalnej paczce. Dla zapytań wystarcza standardowa koperta z `status`, `errorType` i `error`. Powody odrzucenia out-of-order rozróżnia się jako `sealed_range` i `out_of_order_window`.

Globalny semafor dopuszcza do 8 żądań zapisu łącznie: dekodowanych, oczekujących i przetwarzanych. Adapter rezerwuje miejsce przed zbuforowaniem ciała. Wewnątrz limitu pula wykonuje domyślnie cztery operacje równolegle; kolejka przekazuje zadania FIFO, ale nie narzuca kolejności zakończenia rozłącznych paczek. Zajęte miejsca skutkują od razu 503 z `Retry-After:1`. Oczekiwanie na pamięć nie utrzymuje blokad potrzebnych flush. Limit ośmiu przyjęć obejmuje również żądania oczekujące na Written, choć nie zajmują już one miejsca w puli części pamięciowej. Zapytania korzystają z osobnego semafora dwóch miejsc, również bez nieograniczonego oczekiwania.

Generator ponawia 503 z wykładniczym opóźnieniem od 1 s do 30 s i losowym odchyleniem. Nie ponawia automatycznie błędów walidacji, konfliktu i zamkniętego zakresu. Po błędzie połączenia dopuszcza niepewność wyniku opisaną wcześniej.

9 Technologie, konfiguracja i środowisko

9.1 Wybór technologii

Wybór	Uzasadnienie, alternatywy i ograniczenia
Rust 2024	Kontrola danych i własności pamięci; zamiast C++/Go. Koszt nauki, konieczne własne limity alokacji.

Wybór	Uzasadnienie, alternatywy i ograniczenia
Axum 0.8, Tokio 1	Routing, ekstraktory i runtime [9]; alternatywy: Actix Web/Hyper. Blokujące I/O poza wątkami sieci.
Serde 1, serde_json 1	Typowane DTO zamiast ręcznego JSON; własne limity i wykrywanie powtórzonych kluczy.
time 0.3, Clap 4	RFC 3339 i jeden sposób konfiguracji CLI; alternatywy: chrono/plik TOML.
std::fs, libc 0.2, własny kodek	Własny silnik jako cel projektu, jawne/testowalne formaty; zamiast RocksDB/SQLite, kosztem odpowiedzialności za recovery.
Arc, RwLock, Mutex, kanały	Blokady Series, bariera head, WAL/watch; pula pamięciowa i async wait. Bez spinlocków i map lock-free.
crc32fast 1, thiserror 2	Gotowy CRC i typowane błędy zamiast własnego algorytmu i błędów tekstowych.
tracing 0.1, tracing-subscriber 0.3	Zdarzenia i czasy operacji zamiast własnego loggera; bez pełnych treści pomiarów.
Grafana OSS 13.2.0	Wykresy, wybór czasu i provisioning [8]; zamiast własnego GUI, zgodność do sprawdzenia.
Compose v2, Linux	Sieć, usługi i wolumeny [12]; zamiast ręcznej instalacji/Kubernetes, bez orkiestracji klastra.
cargo test, Tower 0.5, generator Rust/reqwest 0.12	Testy modułów, HTTP i obciążenia w jednym stosie, stałe ziarno generatora; zamiast zewnętrznego frameworka.

To wybrane linie zgodności, bez zmiany zależności prototypu. Implementacja utrwala wersje w `Cargo.lock`, toolchain w `rust-toolchain.toml`; demo przypina `grafana/grafana:13.2.0` oraz zapisuje digest. Walidacja: Linux x86-64; inne systemy przez kontenery Linux.

9.2 Konfiguracja procesu

Parametry WAL mapują się na WalConfig brancha; limit W-03 i timeout należą do planowanej integracji. Źródłem konfiguracji są argumenty CLI; brak parametru oznacza wartość domyślną. Compose przekazuje je jako tablicę `command`. Nie wprowadza się równoległych zmiennych środowiskowych dla tych samych ustawień. Zmiany wymagają restartu.

Parametr	Domyślnie	Znaczenie i walidacja
`--listen`	`127.0.0.1:8080`	Adres HTTP; w kontenerze jawnie `0.0.0.0:8080`.
`--data-dir`	`./data`	Katalog dostępny do zapisu, jeden proces na katalog.
`--buffer-duration`	`30m`	Czas do zlecenia flush; dodatnia pojedyncza jednostka czasu.
`--write-workers`	4	Liczba zadań zapisu wykonywanych równolegle, od 1 do 8; nie zmienia liczby logicznych WAL.
`--wal-durability`	`none`	None/Written per proces; żądanie może nadpisać.
`--wal-flush-interval`	`100ms`	Dodatni flush_interval bez waiterów.

Parametr	Domyślnie	Znaczenie i walidacja
`--wal-flush-threshold`	4194304	4 MiB, dodatni próg pobudzenia flushera.
`--wal-buffer-capacity`	8388608	8 MiB początkowej pojemności każdego z dwóch Vec; nie limit.
`--wal-group-delay-us`	200	commit_delay w mikrosekundach, nieujemny.
`--wal-commit-bytes`	262144	256 KiB; od tego rozmiaru pomijane opóźnienie grupy.
`--wal-sync-interval`	`1s`	Dodatni okres syncera, bez oczekiwania Synced klienta.
`--wal-writeback`	`true`	Linux sync_file_range; false wyłącza optymalizację.
`--wal-segment-bytes`	134217728	Dodatni miękki próg 128 MiB, sprawdzany przed kolejną porcją.
`--max-wal-buffer-bytes`	33554432	Planowana W-03: 32 MiB na obie pojemności; co najmniej ich suma i miejsce na przyjmowaną ramkę, bez nieograniczonej kolejki.
`--wal-wait-timeout`	`30s`	Dodatni timeout Written w adapterze; bez rollbacku.
`--max-buffer-bytes`	536870912	512 MiB; dodatni, co najmniej 65536 B. Wartość 0 wyłącza limit próbek, lecz nie pozostałe limity.
`--out-of-order-window`	`5m`	Nieujemny, nie większy od czasu buforowania. Nadal obowiązuje $t > S_s$.
`--max-future-skew`	`60s`	Dopuszczalne wyprzedzenie zegara, nieujemne.
`--lookback-delta`	`5m`	Dodatni zakres wyboru ostatniej próbki.
`--max-series`	10000	Maksymalna liczba szeregów w katalogu; liczba dodatnia.
`--max-metadata-bytes`	67108864	64 MiB na katalog i indeksy, z uwzględnieniem kopii katalogu dla checkpointu.
`--query-timeout`	`30s`	Dodatni limit pojedynczego zapytania.
`--log-level`	`info`	Jeden z: error, warn, info, debug, trace.

Domyślne, stałe limity ochronne z poprzednich sekcji tworzą profil v1: 4 MiB wejścia, 10 000 punktów paczki, 8 przyjętych żądań zapisu, dwa zapytania, 64 MiB na zapytanie i 8 MiB całej ramki WAL. Łączny budżet surowych ciał i zdekodowanych paczek oraz przygotowanych ramek WAL wynosi 64 MiB. Obejmuje także paczki aktualnie przetwarzane. Pamięć jest rezerwowana przed dekodowaniem i przy zmianie reprezentacji; przekroczenie budżetu daje 503 przed enqueue. Osobne 32 MiB WAL obejmuje kopie już przekazane do bufora i flushera. Odczyt ciała ma timeout 10 s.

Limit metadanych sprawdza się przed rejestracją szeregu oraz przed utrwaleniem kolejnej paczki, uwzględniając miejsce na indeks następnego zrzutu. Brak miejsca odrzuca nową paczkę, zamiast przyjmować dane, których zaplanowany flush przekroczyłby limit. Operator może podnieść limit po restarcie, co zmienia założenia zużycia RAM. Wyłączenie limitu bufora nie wyłącza kontroli przepełnień ani błędów alokacji.

9.3 Sieć, trwałość i granica dostępu

Topologię, porty, DNS i montowania pokazuje rys. 2. Publikacje Compose: `127.0.0.1:3000:3000` i `127.0.0.1:8080:8080`. Baza działa bez root z prawem zapisu do własnego wolumenu. Grafana wymaga logowania; hasło administratora dostaje przez nieśledzony plik środowiskowy. API v1 nie ma auth/TLS: granicą jest lokalny host i zaufana sieć demo; wyjście poza nią wymaga osobnej warstwy dostępu. Przeglądarka nie łączy się z TSDB, więc CORS nie jest potrzebny.

Compose czeka na `/readyz` przed startem generatora. Po restarcie generator ponawia połączenie, Grafana odzyskuje odczyt pod stałym adresem. Zatrzymanie kontenerów zachowuje wolumen; jego usunięcie jest osobną operacją operatora.

9.4 Diagnostyka

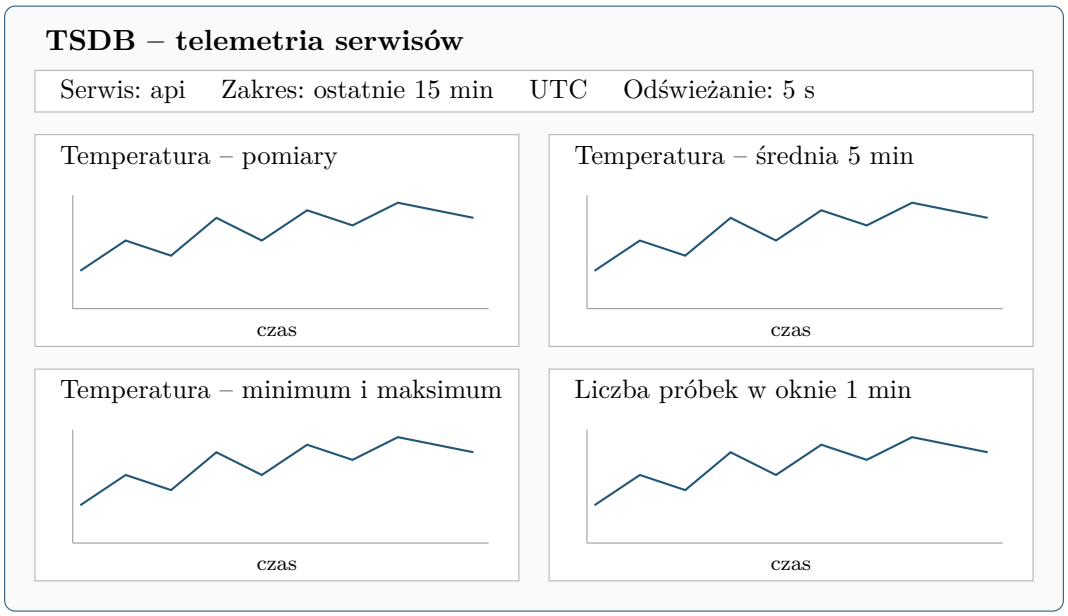
Logi żądań: request ID, endpoint, wynik, czas, lokalny LSN, None/Written i liczniki punktów; identyfikator uruchomienia odróżnia powtarzalne LSN po restartach. Flush: generacja, wal_checkpoint, punkty, bajty, czas cut i zapisu segmentu. RecoveryReport: pliki, rekordy, obcięcie oraz late_series, unknown_series, dropped_samples. Bez pełnych ciał i etykiet w każdym logu.

Pomiary: RSS, head, rzeczywiste pojemności obu Vec WAL, backlog_bytes, waiterzy, write, fdatasync, writeback, pauza cut, kolejka i flush. Syncer nie udostępnia granicy synced. Liczniki mogą trafiać okresowo do logów, bez dodatkowego systemu monitoringu.

10 Wizja interfejsu użytkownika i demonstracja

10.1 Dashboard Grafany

Dashboard „TSDB – telemetria serwisów” zawiera wybór jednego serwisu, selektor czasu, automatyczne odświeżanie co 5 s i cztery panele. Początkowy zakres to ostatnie 15 min, strefa wyświetlania UTC. Źródła generatora mają etykiety `service` i `instance`; różne instancje są osobnymi liniami na wykresie.



Rysunek 12: Makieta układu dashboardu. Linie są ilustracją, nie wynikami eksperymentu.

Panel	Wyrażenie projektowanego dashboardu
Pomiary	`temperature_celsius{service="$service"}`
Średnia 5 min	`avg_over_time(temperature_celsius{service="$service"}[5m])`
Minimum i maksimum	Dwa zapytania: `min_over_time(temperature_celsius{service="$service"}[5m])` oraz `max_over_time(temperature_celsius{service="$service"}[5m])`.
Liczba próbek	`count_over_time(temperature_celsius{service="$service"}[1m])`

Zmienna `service` jest typu *Custom*, z wartościami `api` i `worker`. Wyłączone są opcje wielokrotnego wyboru i „All”, aby Grafana nie generowała regexów. Zapytania są zapisane w trybie *Code*. Minimalny krok panelu to 1 s; Grafana może go zwiększać dla dłuższego zakresu. Nie używa się makra generującego funkcje spoza podzbioru.

Brak danych jest przedstawiany jako przerwa lub „No data”, bez łączenia luk wypełnieniem zerami. Błąd API pozostaje widocznym błędem panelu, a nie pustym poprawnym wynikiem. Panel liczby próbek nie jest licznikiem wszystkich szeregów ani agregacją między instancjami.

10.2 Profil standardowego źródła Prometheus

Provisioning ustawia stały UID, adres bazy i metodę POST. Poniższy fragment jest projektowaną konfiguracją do użycia podczas implementacji demo; znaczenie ustawień źródła wynika z dokumentacji Grafany [7].

```
apiVersion: 1
datasources:
  - name: TSDB
    uid: tsdb
    type: prometheus
    access: proxy
    url: http://tsdb:8080
    isDefault: true
    editable: false
    jsonData:
      httpMethod: POST
      timeInterval: 1s
      manageAlerts: false
      disableRecordingRules: true
      prometheusType: Prometheus
      prometheusVersion: 2.0.0
```

`prometheusVersion:2.0.0` wybiera konserwatywny profil funkcji klienta, nie deklaruje wersji silnika TSDB. Prawdziwą wersję zwraca `buildinfo`. Wyłącza się incremental querying, exemplary i funkcje edytora wymagające nieobsługiwanych endpointów. Provisioning dashboardu wskazuje katalog plików JSON i UID `tsdb`. Brakującą funkcję Grafany weryfikuje się na przypiętym obrazie, bez deklarowania zgodności ze wszystkimi wersjami.

10.3 Scenariusz demonstracji

1. Uruchomić Compose z pustym lub istniejącym wolumenem. Poczekać na gotowość bazy i uruchomienie generatorów.
2. Generatory `api` i `worker` wysyłają co sekundę temperaturę dwóch instancji. Jeden profil celowo wprowadza opóźnione punkty.
3. Otworzyć dashboard, zmienić serwis i zakres czasu; porównać pomiary z agregacjami i liczbą próbek.

4. W profilu demonstracyjnym skrócić buforowanie do 1 min, a okno opóźnienia do 30 s. Pokazać potwierdzane zapisy i odczyty podczas flush, niezmienność wcześniejszych danych po podmianie reprezentacji oraz błąd punktu skierowanego do zamrożonego zakresu.
5. Zrestartować bazę z zachowaniem wolumenu. Pokazać wcześniejsze dane i dalszy napływ pomiarów.

Profil pokazowy z krótkim buforem nie jest profilem testu zasobów z lab1. Scenariusz awarii z SIGKILL jest wykonywany w testach kontrolowanych; sam restart demonstracyjny nie dowodzi gwarancji NFR-04.

11 Weryfikacja architektury i kryteria odbioru

11.1 Powiązanie z wymaganiami

Wymaganie	Mechanizm	Weryfikacja
FR-01	HTTP, walidacja, atomowa ramka WAL.	T-01: zapis pojedynczy i paczka; błędna paczka nie zmienia danych.
FR-02	Kanoniczny klucz i katalog szeregów.	T-02: kolejność etykiet, inne wartości, puste etykiety i kolizje hash.
FR-03	Written, bufor WAL i checkpoint.	T-03, T-10, T-16: replay potwierdzonych Written; osobna ocena None.
FR-04	Siatka czasu i jeden ReadView.	T-04: lookback, granice, krok niedzielący zakresu i puste wyniki.
FR-05	Parser ograniczonej gramatyki.	T-05: wszystkie konstrukcje, pełne zużycie wejścia, odrzucane rozszerzenia.
FR-06	Agregacje osobno dla szeregu.	T-06: pięć funkcji, granice okna, kilka szeregów, tolerancja błędu f64.
FR-07	API Prometheus i provisioning.	T-07: dashboard i test połączenia na przypiętym obrazie Grafany.
FR-08	Rotacja bloków, budżet pamięci i flush w tle.	T-08, T-14–15: timer, nowe zapisy podczas flush i pamięć utrzymywana przez odczyt.
FR-09	W, H_s, S_s.	T-09: granice równości, flush, zmiana konfiguracji i restart.
NFR-01–03	Paczki, indeksy, limity i kontrola alokacji.	P-01–03: testy HTTP, przepustowości i RSS z opisanym profilem.
NFR-04	Recovery i kolejność publikacji plików.	T-10 i P-04: wymuszone awarie i czas gotowości.
NFR-05	Typowane błędy i walidacja.	T-11: macierz błędnych żądań bez zakończenia procesu.
DA-02–04, DA-08	Index, blokady Series, bariera rotacji i jeden WAL.	T-12–15 i P-05: równoległość, brak zakleszczeń i poprawność widoków.
DA-07, DA-09	Flusher, syncer, granice prefiksów i tryby per żądanie.	T-16–20 i P-06: przyczynowość definicji, rotacja, checkpoint i błędy I/O.

11.2 Scenariusze poprawności

T-01: Jeden punkt, wieloszeregowa paczka i 10 000 punktów: odczyt po sukcesie widzi całość. Błędny punkt nie zmienia próbek/katalogu; niewidoczne luki ID dozwolone.

T-02: Kolejność etykiet nie zmienia Series, wartość zmienia; puste usuwane, powtórzone klucze JSON/nazwy zastrzeżone odrzucane, kolizja hash nie scala kluczy. Adapter dodaje wewnętrzne ___name___.

T-03: Restart z WAL, samymi segmentami/pustym nowym WAL i stanem mieszanym: te same próbki/etykiety/S_s, mimo zmiany head ID. Nowy plik ma rosnące ID, pierwszy LSN jest liczony od 0.

T-04: Instant/range: start=end, krok=0, odwrócony zakres, koniec poza siatką, dokładna granica lookback, wiele Series/brak danych.

T-05: Parser: funkcje, escape, whitespace, 1+1; błędna jednostka, trailing input, regex i nieobsługiwana funkcja.

T-06: Ręczne wyniki, puste/brzegowe $(T - R, T]$, skrajne wejścia i nieskończony wynik; RAM/dysk/mieszany widok z tolerancją $10^{-9} \max(1, |wynik_{\text{ref}}|)$.

T-07: Przypięta Grafana: provisioning, wszystkie panele, wybór czasu/serwisu, przerwy, restart i test połączenia.

T-08: Flush czasowy/pamięciowy przy odczycie: punkt raz, bez pełnej kopii RAM; za mały budżet paczki nie zapętla flush.

T-09: Równość i obie strony $H - W$, S, duplikaty/konflikty, ponowna walidacja po cut/freeze. Zmiana W nie odrzuca replay.

T-10: ≥ 20 awarii procesu w Written, z osobnym dziennikiem potwierdzeń klienta: środek ramki, po write/sync, podczas cut, rename segmentu/manifestu, przed/po usuwaniu WAL. Potwierdzone Written wracają; None i paczki bez odpowiedzi mogą wrócić w całości lub być nieobecne. Drop/close nie zastępuje SIGKILL.

T-11: Błędne typy/NaN, rozmiar, parametry (w tym synced), pełne przyjęcia/zapytania i timeout Written: właściwy kod/opis, dalsza diagnostyka. Timeout nie wycofuje publikacji.

T-12: Co najmniej dwa zapisy rozłącznych Series. Wstrzymać flusher po head append klienta Written: inny None, także do tego Series, oraz odczyt mogą skończyć. Konkurencyjne duplikaty/konflikty wspólnego Series bez utraty próbek.

T-13: Paczki tych samych Series w odwrotnej kolejności wejścia bez deadlock; odczyt widzi całość albo stan sprzed paczki. Rejestracja jednego klucza daje jeden Series; zmiana wersji katalogu podczas snapshot wymusza ponowienie.

T-14: Wstrzymać flush po cut/freeze: nowy active działa, obie generacje czytelne. Pierwszy zapis istniejącego Series w nowym g ponownie zawiera definicję; usunięcie WAL $\leq N$ nie psuje restartu. Powtórzyć dla Series utworzonego podczas flush.

T-15: Wolny flush, active bliski limitu i ReadView starego bloku: RAM rozliczony do ostatniego uchwytu, 503 przed append, bez kolejki frozen i blokowania zwalniania pamięci.

T-16: Przy zatrzymanym flusherze None odpowiada, Written czeka mimo widocznego head; write zwalnia Written. Okresowy sync nie tworzy dodatkowego trybu/warunku odpowiedzi. Duplikat None ponowiony jako Written rzeczywiście czeka.

T-17: Powtórne definicje i różne ID tego samego LabelSet mapują się na jeden Series. Próbki przed definicją wracają po jej dostarczeniu. Resolver raportuje definicję, która nigdy nie nadeszła, a gotowość produkcyjna jest wtedy zablokowana.

T-18: Written nie zależy od długiego flush_interval: waiter budzi flusher. Postęp watch przed subskrypcją, krótki write, anulowanie i zamknięty kanał nie powodują utraty pobudzenia/fałszywego sukcesu. Rotacja z równoległym sync zachowuje ważny Arc<File>.

T-19: Wstrzymany head append opóźnia barierę cut, waiter Written nie. Cut nie obejmuje nowych append po zwolnieniu bariery. Po manifeście pozostawione WAL $\leq N$ są pomijane. Rotacje rozmiarowe wewnątrz generacji i przekroczenie miękkiego progu o porcję nie zmieniają replay; brak dzielenia ramki.

T-20: Budżet active+spare i scratch wymusza backpressure przed append, bez części paczki. Błąd write/sync/writeback kończy proces bez sukcesu nieosiągniętych Written. Jawny close zapisuje również None, synchronizuje i odrzuca późniejsze append. Wcześniejsze Written wracają przy sprawnym nośniku.

Kodeki: zgodność bajtowa RecordEncoder, 9-bajtowy nagłówek, len payloadu, CRC(kind || len || payload), każde ucięcie małej ramki, nadmierne liczniki/offsety i pełna błędna ramka na końcu (błąd startu). Bariery wymuszają przeploty. Sprawdzić puste pliki, luki po N, replays bez usunięcia starego WAL, restart LSN=0 i brak trwałych head ID.

11.3 Profil pomiarów wydajności i zasobów

Raport podaje CPU/rdzenie, RAM, dysk/system plików, OS, toolchain, commit, konfigurację i współbieżność klienta. Referencja: Linux x86-64, ≥ 4 rdzenie, 8 GiB RAM, lokalny SSD bez innego obciążenia, build release; parametry nie zastępują pomiaru.

Test	Przebieg	Cel z lab1
P-01	Po 30 s rozgrzewki co najmniej 5 min pomiaru. Osobne scenariusze zapisu jednego punktu, paczki 1000 punktów, odczytu 1000 wyników i agregacji. Profil bazowy Written; czas HTTP obejmuje kolejkę, publikację head i oczekiwanie na write.	p95 zapisu punktu $\leq$ 100 ms; paczki $\leq$ 500 ms; odczytu $\leq$ 200 ms; agregacji z RAM $\leq$ 500 ms.
P-02	1000 szeregów o stałych etykietach, paczki po 1000 punktów, 4 współbieżnych klientów, Written. Co najmniej godzina, by objąć flush; raportować także najgorsze okna 1 s i 10 s.	Średnio $\geq$ 10 000 nowych potwierdzonych punktów/s, z uwzględnieniem rotacji, działania flush i błędów.
P-03	10 000 punktów/s, 1000 szeregów, buforowanie 30 min, domyślne limity, minimum 65 min. Bazowy scenariusz bez zapytań, dodatkowo wariant z dwoma zapytaniami.	RSS procesu $\leq$ 1 GiB, w tym szczyt przy flush i realokacji. Raportować rzeczywisty czas przebywania punktów w RAM.
P-04	Milion potwierdzonych punktów; osobno stan z przewagą WAL i stan po flush. Pomiar od startu procesu do pierwszego 200 z readyz; osobno cache zimny i ciepły.	Gotowość w $\leq$ 10 s i brak utraty danych w T-10.
P-05	Warianty 1, 2, 4 i 8 zadań zapisu i klientów. Osobno rozłączne szeregi oraz wszystkie paczki kierowane do jednego Series. Pomiar obejmuje co najmniej jeden flush.	Ocena rzeczywistej równoległości, kosztu blokad i wspólnego WAL; bez założenia liniowego przyspieszenia.

Test	Przebieg	Cel z lab1
P-06	Osobne przebiegi None i Written dla tych samych paczek. Zmiana opóźnienia grupy i obserwacja opóźnień flushera/syncera.	Raportować p95, przepustowość i bajty oczekujące oddzielnie; None nie zalicza kryterium trwałej przepustowości.

P-01: selektor jednego Series, 1000 chwil co 1 s i próbki co 1 s; agregacja tego Series, okno 5 min, ta sama siatka, potrzebne dane w RAM. P-02/03: 1000 Series po 10 punktów/s, najwyżej cztery krótkie etykiety; bazowo 0% out-of-order, dodatkowo 1%/10% z opóźnieniem do 5 min. Odrzuceń po zamknięciu nie wlicza się do przepustowości.

Surowe próbki z 30 min: $10\,000 \cdot 1800 = 18$ mln, $18\,000\,000 \cdot 16 = 288\,000\,000$ B $\approx 274{,}7$ MiB. Dochodzi zapas pojemności, active/frozen, uchwyty, metadane, przyjęcia, scratch, HTTP i zapytania. Limit head 512 MiB daje margines, **nie dowodzi RSS ≤ 1 GiB**. Mierzy się też wspólny WAL, oczekiwanie na Series, barierę z I/O cut i osobno dyskowy flush segmentu poza barierą. Nieosiągnięty cel raportuje się z wynikiem i wąskim gardłem.

11.4 Ryzyka i warunki akceptacji projektu

Ryzyko	Sposób ograniczenia	Obserwowany wskaźnik
Flush zużywa zapas pamięci	Rotacja przy 80% budżetu, jeden flush w tle, rozliczanie bloków podtrzymywanych przez odczyty i backpressure.	Czas flush i bariery, pamięć obu generacji, 503 i p95 zapisu.
Konkurencja o blokady i WAL	Krótkie blokady Series, enqueue bez I/O i oczekiwanie na Written dopiero po publikacji.	T-12–13, P-05–06, czas części pamięciowej, wait Written i pauzy cut.
Koszt out-of-order	Wektor uporządkowany, jawny limit W i zamknięta granica; test udziału opóźnionych danych.	Czas wstawienia i liczba przesuniętych próbek.
Wzrost indeksów i plików	Limit katalogu i metadanych; jawny brak retencji i kompaktowania w v1.	Liczba szeregów, segmentów, bajty metadanych i dysku.
Niepełna zgodność Grafany	Ustalony obraz, tryb Code, ograniczone zmienne i wczesny test rzeczywistych żądań.	T-07 oraz lista wywołanych endpointów i parametrów.
Błąd checkpointu	Manifest jako jedyne źródło prawdy, sync przed rename, WAL usuwany na końcu.	T-10 i testy uszkodzeń plików.

Akceptacja projektu architektury wymaga spójności kontraktów i formatów, pokrycia wszystkich obszarów wytycznych lab2 oraz prześledzenia scenariuszy awarii. Akceptacja późniejszej implementacji wymaga wykonania testów i pomiarów; niniejszy dokument nie oznacza ich zaliczenia.

A Słownik

Pojęcie	Znaczenie
Head / Index / Series	Pamięciowy katalog i active/frozen / jeden katalog kluczy i uchwytów / szereg z własną blokadą i blokami.
LabelSet / SeriesId	Kanoniczne etykiety z ___name___ / lokalne ID head/WAL, odtwarzane przez mapowanie po kluczu.
WAL / LSN	Bufor logu przed head append / liczba bajtów przyjętych w bieżącym Wal::open, reset po restarcie.
Flusher / Syncer	Zapisuje porcje, publikuje Written i obsługuje cut / okresowo sync_data, bez potwierdzenia per żądanie.
Cut / Checkpoint	Drain, sync i rotacja WAL zwracające zamknięty plik N / trwały manifest z danymi wszystkich WAL $\leq N$.
Generacja / logged_gen	Grupa wspólnie zamrażanych bloków / znacznik, czy Series ma definicję w aktualnej generacji WAL.
Segment / Manifest	Niezmienny plik kluczy, próbek i indeksu / spis segmentów, klucze, granice zamknięcia i wal_checkpoint.
Freeze / Flush	Przeniesienie active do frozen po cut / zapis segmentu i manifestu w tle, potem remove_through.
Resolver / ReplaySink	Mapowanie ID i pending late series / odbudowa Index/head po LabelSet.
Out-of-order / S_s	Próbka starsza od najnowszej / granica zamknięcia: nowy czas musi być większy.
Lookback / Provisioning / RSS	Maksymalny wiek próbki selektora / deklaratywna konfiguracja Grafany / rezydentna pamięć procesu.

B Budowanie dokumentu

Źródło jest samodzielnym plikiem LaTeX. Diagramy i makieta są zapisane w TikZ, bez zależności od zewnętrznych obrazów lub usług. Wymagane są pdfLaTeX, latexmk oraz pakiety użyte w preambule, w szczególności obsługa języka polskiego i TikZ. Z katalogu głównego repozytorium:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error \
  -outdir=target/lab2-doc \
  pz/lab2/lab2-Blonowski-Zaborski.tex
```

Pliki pomocnicze trafiają do ignorowanego katalogu `target/lab2-doc`. PDF do oddania jest kopią wyniku kompilacji umieszczoną obok źródła. Po zmianach należy sprawdzić ostrzeżenia składu, odwołania, spis treści oraz czytelność tabel i diagramów.

Literatura

[1] H. Błonowski, J. Zaborski, *Etap 1: Specyfikacja wymagań i zakres pracy*, wersja 1.1, 21.10.2026, dokument projektu w `pz/lab1`.

[2] Repozytorium projektu, gałąź `23-wal-improvements`, commit `e1354b9`, pliki `src/storage/wal`, `src/storage/series.rs`, `src/db.rs`; odczyt lokalny 02.11.2026. `https://github.com/janzaborski/rust-tsdb/tree/e1354b9/src/storage/wal`.

[3] T. Pelkonen, S. Franklin, P. Cavallaro, Q. Huang, J. Meza, J. Teller, K. Veeraraghavan, *Gorilla: A Fast, Scalable, In-Memory Time Series Database*, Proceedings of the VLDB Endowment, 8(12), s. 1816–1827, 2015, sekcje 4.2–4.3, `https://www.vldb.org/pvldb/vol8/p1816-teller.pdf`.

[4] Prometheus, *HTTP API*, https://prometheus.io/docs/prometheus/latest/querying/api/.

[5] Prometheus, *Querying basics*, https://prometheus.io/docs/prometheus/latest/querying/basics/.

[6] Prometheus, *Query functions*, https://prometheus.io/docs/prometheus/latest/querying/functions/.

[7] Grafana Labs, *Configure the Prometheus data source*, https://grafana.com/docs/grafana/latest/datasources/prometheus/configure/.

[8] Grafana Labs, *Provision Grafana*, https://grafana.com/docs/grafana/latest/administration/provisioning/.

[9] Projekt Axum, *Dokumentacja biblioteki*, https://docs.rs/axum/latest/axum/.

[10] Projekt Rust, *std::fs::File*, https://doc.rust-lang.org/std/fs/struct.File.html.

[11] Projekt crc32fast, *Dokumentacja biblioteki*, https://docs.rs/crc32fast/latest/crc32fast/.

[12] Docker, *How Compose works*, https://docs.docker.com/compose/intro/compose-application-model/.

[13] Linux man-pages, *write(2)*, https://man7.org/linux/man-pages/man2/write.2.html.

[14] Linux man-pages, *fsync(2), fdatasync(2)*, https://man7.org/linux/man-pages/man2/fsync.2.html.

[15] Linux man-pages, *sync_file_range(2)*, https://man7.org/linux/man-pages/man2/sync_file_range.2.html.

[16] Projekt Tokio, *tokio::sync::watch*, https://docs.rs/tokio/latest/tokio/sync/watch/index.html.

Dokumentację zewnętrzną sprawdzono 24.10.2026. Zachowanie własnego silnika określa niniejsza specyfikacja; odnośniki *latest* służą jako kontekst, a zgodność integracji sprawdza się z przypiętymi wersjami używanymi w demonstracji.